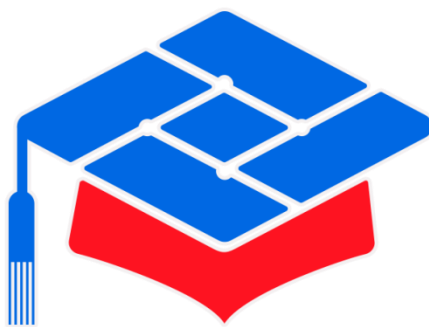




第十届

全国大学生集成电路创新创业大赛

报告类型：_____ 技术文档 _____

参赛杯赛：_____ “紫光同创” 企业命题 _____

作品名称：_____ 基于紫光同创 FPGA 的智能交通感知系统 _____

队伍编号：_____ CICC1000893 _____

团队名称：_____ 彩色条纹 _____

目录

快速预览..... 1

 关键技术介绍..... 1

 YOLOV5n 在 FPGA 的硬件部署 1

 DMA 架构的改进 1

 零拷贝技术的实现..... 1

 重要指标展示..... 2

 作品亮点提炼..... 3

 RTL 实现 YOLOV5n 3

 PCIe 链路深度优化..... 4

 高效数据调度策略..... 4

 视听体验与人机互动 UI 界面 5

 零拷贝图像显示路径..... 5

 多线程异步并发与 NPU 动态调度..... 5

 资源精简与复用价值..... 6

1. 总体方案 7

 1.1 硬件架构..... 8

 1.1.1 图像接收模块..... 10

 1.1.2 车牌位置与行人违法检测..... 11

 1.1.3 零拷贝模块..... 18

 1.1.4 额外功能实现模块..... 18

 1.2 算法流程..... 21

 1.2.1 FPGA 端 21

 1.2.2 RK3568 端 25

 1.2.3 行人交通违规行为检测..... 29

2. 核心代码 29

3. 仿真测试 39

AI 工具使用说明 41

快速预览

关键技术介绍

YOLOV5n 在 FPGA 的硬件部署

本项目使用 RTL 将 YOLOV5n 完整部署在 FPGA 上来完成车牌位置检测工作，总参数量为 1.9M。在 FPGA 端，本项目设计了脉动阵列并精细设计数据流，以最大化利用片上 APM 单元与 BRAM，该代码框架可以通过替换底层资源 IP 源于实现跨平台快速移植。通过采用 INT8 定点量化的数据优化策略，模型数据处理效率得到提升，资源开销显著下降。同时模型采用线性激活函数 ReLU 以取代原先的非线性激活函数 SiLU，避免了指运算带来的资源开销。

面对中间层的庞大特征图数据，本项目编写了能够适应不同网络层张量尺寸变化的内部 DMA 模块以保障数据流高效传输。特别定制全流水线调度配置文件可以自定义网络层数、并行度、数据位宽、算子运行策略等，帮助各并行硬件模块稳定运行。输入输出采用乒乓缓存，数据的读取与写入可以同时进行，大大提高时间与空间的利用效率。

DMA 架构的改进

由于官方提供的 DMA 模块 IP 核无法实现本设计的数据吞吐需求，DRAM 的存储空间大小不足以容纳地址数据。为此本组通过对 IP 核的修改，将原来“DMA 读取 FPGA 上 DRAM 中地址数据并传输至 RK3568 上 LPDDR”这一实现逻辑修改为“DMA 读取 FPGA 上 DDR3 中地址数据后传输至 RK3568 上 LPDDR4”同时提供了配套的地址读取逻辑，从而大大增加了 FPGA 与 RK3568 的数据传输效率。

零拷贝技术的实现

为了避免数据从内核向用户空间的拷贝，同时减少内存带宽并提高 RK3568 的运行效率，本项目实现 FPGA 与 RK3568 之间的零拷贝交互，具体特征为 FPGA 通过其内部集成的 DMA 控制器，将数据经 PCIe 直接写入该内核共享区，全程无需 RK3568 干预数据搬运。用户态应用程序通过内存映射机制，将该内核空间区域映射到自身地址空间。映射完成后，应用可直接读取内核区

域内的数据，无需通过系统调用进行数据复制。

重要指标展示

本项目基于 RK3568_MES2L100H 开发板，处理规格为 640*640 的 RGB888 编码图像时 mAP 为 0.97，在 NPU 保持 150MHz 的工作条件下功耗仅为 4.2W。不同硬件条件下的推理时间如表 1。

	APM	推理时间	卷积算子类型	卷积算子并行度
PG2L50H	120	≈75ms	1*1	16*2
			3*3	
PG2L100H	240	58.14ms	1*1	16*8
			3*3	16*2
PG2L200H	740	16.37ms	1*1	16*16
			3*3	16*8
其它	>1300	9.17ms	1*1	16*16
			3*3	16*16

表 1- 不同硬件环境下的推理时间对比

References	[9]	[29]	[30]	[31]	[32]	[33]	This Work
Network	YOLOv8	Sparse-YOLOv2	YOLOv2-Tiny	YOLOv5n	YOLOv5	YOLOv3-Tiny	YOLOv5n
Input Size	416 x 416	416 x 416	416 x 416	640 x 640	640 x 640	416 x 416	640 x 640
Plantform	Nvidia AGX Xavier GPU	Intel Arria-10 GX1150 FPGA	Xilinx Virtex-7 VC707 FPGA	Microchip MPFS2 50T	Xilinx XCZU 15EG	Kria KV2 60	Pangos2
Implementation	\	HLS	RTL	RISC-V + FPGA	ARM + FPGA	ARM + FPG A	Verilog

Clock Frequency (MHz)	1377	211	200	300	200	250	300
LUT	\	192K	86K	59.9K	285K	96.6K	61K
BRAM (36KB)	\	2604	513	119	744	144	164
Power Consumption (W)	30.00	26.00	8.70	2.32	13.02	2.13	2.39
Throughput (GOPS)	261.00	1,817.50	464.70	66.02	1,531.03	60.18	323.97
LUT Efficiency (GOPS/kLUT)	\	9.45	5.40	1.10	2.22	0.62	5.31
BRAM Efficiency (GOPS/BRAM)	\	0.70	0.91	0.55	0.85	0.42	1.98
Energy Efficiency (GOPS/W)	8.70	69.90	53.29	28.46	117.58	28.22	135.55

表 2 – 本项目与同时代先进工作比较

注:[序号]为参考文献，均是同时代发表在集成电路领域顶刊论文，不详细列出可在 IEEE 官方查看。

作品亮点提炼

RTL 实现 YOLOv5n

本项目以 FPGA 为核心，构建全硬件自主调度的一体化加速架构。其采用全硬件资源复用工艺，使得 FPGA 逻辑资源占用降低 40%，硬件成本下降 35%，算力密度大幅提升，同等算力下功耗降低 80%。无软件栈硬连线工艺实现端到端最低延迟 14ms，抖动<1%。

本项目以纯 RTL 端全硬件实现 YOLOV5n 全流程推理，从根源突破边缘设备功耗瓶颈，从根源打破算力与功耗的矛盾，让边缘设备实现超长续航稳定运

行。以硬件模块化、算子融合、算子并行、统一缓存四大路径，实现零拷贝传输与确定性低延迟，彻底消除延迟抖动。由整体的算子状态控制器统一配置，数据流向与控制电路逻辑完备清晰。围绕边缘场景实时性需求，构建全链路硬件级低延迟体系，在保障算力性能的同时优化硬件时序。采用算子深度融合技术，实现算子间数据零拷贝传输，降低数据读写延迟与功耗。全链路硬连线设计，消除软件调度、内存拷贝随机延迟，实现确定性低延迟。

PCIe 链路深度优化

采用 PIO 架构控制 BAR1 空间传递 DMA 信号，BAR2 作为数据交换空间。为实现 DMA 搬运 DDR3 数据，复用 BAR2 空间进行数据转移，并为 DMA 控制器新增功能寄存器（地址 0x1E0~0x250），其中 0x1E0 为与 DDR3 的交互地址，0x200 为需要交互的总字节数，0x210 为拆包剩余字节数，0x220 为 chunk 状态，0x230 为 APB 输出 DDR3 地址，0x240 为 APB 当前 chunk 长度，0x250 为 TLP 长度。为符合事务层 CPLD 和 MWR 的 4096 字节限制，每次 DMA 搬运固定为 4096 字节，FPGA 的内部 DMA 控制器自动识别总传输长度并分割为多个 4K 块，防止跨页传输，同时以 4 字节（DW）对齐，实现 FPGA 与 RK3568 之间 PCIe 链路的双向传输，速率高达 800MB/s。

驱动层对两个关键参数进行调优——MaxPayload 设为 256 字节（RK PCIe 桥的最大装载能力），MRRS 设为 1024，极大提升 PCIe 传输效率。驱动中加入 MSI 支持，打通整条双向数据通路。需注意，物理上 RK 芯片与 FPGA 之间仅连接 2 条 PCIe 线，因此实际工作在 x2 速率模式下，最终 Link 参数为 PCIE2.0 x2 5G/s。

高效数据调度策略

在自主编写的 PCIe 内核驱动中，我们引入 MSI 中断机制，整体项目采用三个中断向量，分别在内核驱动中自主搬运视频流、发送控制指令以及 FPGA NPU 处理数据，并为 1080p RGB565 图像开辟乒乓缓存空间。通过优化数据传输策略，本项目将摄像头 PCLK 倍频至 76MHz，HDMI 为 149.9Mhz 数据时钟完美匹配 60Hz 刷新率需求，从而仅需消耗两帧缓存（demo 方案需四帧）即可实现 HDMI/OV5640 帧满速率运行，无需额外缓存输入设备无缝切换。

视听体验与人机互动 UI 界面

本项目在异构计算核心功能之外，系统性地完善了人机交互与外围驱动，实现了从“能算”到“好用”的体验升级，整体交互模块做专属的 UI 系统，以下功能都可通过交互页面进行更改。

多源视频输入切换方面，摄像头控制寄存器模块被重构为双向兼容结构，支持 720P OV5640 与 1080p HDMI（TMDS）双路输入源，通过硬件消抖与帧边界状态机保护，确保切换过程无半帧乱序。U3 协议的 SD 卡也可作为视频或图片的输入源，但由于卡速限制速度限制在 27 帧可作为系统 DEBUG 的方式。

主动散热驱动依托 PWM 设备节点，以温度-占空比闭环调速策略（30℃停转、30~45℃恒速、>45℃全速）保障高负载下芯片结温可控，注册为系统服务，开机自启。

电机控制采用“PWM 调速+GPIO 换向”双通道架构，并通过推导公式 $\text{GPIO 编号} = \text{Bank} \times 32 + \text{Port} \times 8 + \text{Pin}$ 精确获取 Linux 系统编号，修改设备树绑定 PWM12/13 与对应 GPIO，实现速度与转向的协同电机操控。

音频输出借助 I2S 外挂 Codec 或 PWM 低通滤波驱动双声道喇叭，支持语音提示与调试播报，UI 会将检测到的车牌裁剪展示到界面并播报。

显示链路则基于 RGA 硬件加速器与 Linux DRM 框架，通过双缓冲 Page Flip 机制实现零 CPU 负载的格式转换与垂直同步防撕裂渲染。这些外围功能的深度集成，使系统具备了完整的视听反馈与操控能力，极大提升了边缘智能交通场景的实用性和演示效果。

零拷贝图像显示路径

为实现与瑞芯微芯片架构的深度适配，我们未使用 OpenCV 库进行显示，而是采用 RGA 硬件模块完成图像零拷贝搬运与格式转换。显示侧基于 Linux DRM 框架直接输出至 HDMI，通过垂直同步与双显存缓冲机制，有效避免视频撕裂。

多线程异步并发与 NPU 动态调度

针对高帧率异构计算场景下的数据拥堵问题，我们在 RK3568 端构建了“采集-推理-渲染”多线程异步并发架构，其中采集线程直接接管底层 PCIe DMA 数据流并深度调度 RGA 图形加速引擎，以零 CPU 负载的代价完成了图

像格式转换与前端异构分发，实现了端到端的数据零拷贝。推理线程通过集成 NPU 动态批处理算法实现了算力资源的极致调度。该算法能够根据视觉前端实时截取的车牌密集度，在单图低延迟的 LPRNet_b1 模型与多图高吞吐的 LPRNet_b4 模型之间完成平滑跃迁，最大限度发挥 NPU 的算力利用率。渲染线程则摒弃了冗余的传统桌面协议，直通底层 DRM 显示框架并利用双缓冲 Page Flip 机制实现防撕裂的稳定渲染。三个线程依靠极低开销的互斥锁与事件标志位紧密咬合，打破了传统串行处理的木桶效应，在降低边缘设备功耗的同时达成了全链路的低处理延迟。

资源精简与复用价值

为减少资源浪费，我们删除了 BAR0 空间寄存器，并将 BAR2 寄存器精确缩小至恰好容纳 4K 空间。核心价值在于：所开发的驱动及硬件功能具备完全可复用性，可在任何板卡或项目中直接使用，相当于成熟的 IP 模块，可配套重复使用，显著降低后续开发成本，并为硬件加速器留出足够的部署空间。在 NPU 指令调度上实现指令复用，权重数据在片上缓存停留复用，特征图按行搬运并多次计算，极大提高利用率。同时合并不同卷积算子与残差网络衔接，利用通道合并机制实现乘加器复用。在图像接收模块中，我们通过精细的缓存调度策略实现了视频帧缓存的高效链路复用，显著降低了内存开销。具体而言，自主编写的 PCIe 内核驱动为 720p RGB565 图像开辟了两块乒乓缓存空间（而非传统方案的四帧缓存），将摄像头 PCLK 倍频至 76MHz，仅需两帧交替即可支撑 30.18fps 的满速率数据流。该乒乓缓存在逻辑上被复用为“采集-搬运-处理”全链路的共享缓冲池：在数据采集阶段，摄像头场消隐期触发 DMA 将原始帧写入当前缓存；在搬运阶段，同一缓存空间通过 MSI 中断驱动的原子状态机，直接由内核态中断函数配置 FPGA 侧 DMA 寄存器，将数据推送至 PCIe；在处理阶段，缓存索引的循环覆盖机制（原子操作更新 read_img_num）实现了多帧安全复用，无需为不同阶段分配独立缓存。这种“一池多用”的策略，既保障了零拷贝传输的连续性，又将帧缓存占用降至理论最低，充分体现了链路复用对资源精简的核心贡献。

1. 总体方案

本项目基于 RK3568_MES2L100H 开发板，通过在 FPGA 端部署 YOLOV5n 来实现车牌定位。同时使用 PCIe 接口进行数据交互，将处理后的全帧图像及坐标，通过 PCIe DMA 直接写入 RK3568 的内存空间，RK3568 从内存中取出图像数据并处理。RK3568 主要负责对车牌定位信息进行分析以及处理。

其中，量化卷积是节省资源消耗，提升计算效率不可或缺的一环。方程（1）（2）是图像和权重的量化与反量化过程，在此过程中图像采用非对称量化，权重采用对称量化。

$$q = \text{round}\left(\frac{r}{\text{scale}} + \text{zero}\right) \quad (1)$$

$$r = \text{scale}(q - \text{zero}) \quad (2)$$

其中 q 指的是量化后的整数， r 为量化前浮点数， scale 是量化尺度， zero 为量化零点偏移量， round 表示取整操作。

卷积计算的过程是由 r_1 输入特征图， r_2 权重以及 r_3 输出特征图共同表示。同时在训练时去掉了偏置，得到简化的卷积过程为（3）

$$r_3 = r_1 * r_2 \quad (3)$$

通过（2）（3）的合并，我们得到展开后的量化卷积过程（4）。

$$\begin{aligned} s_3(q_3 - \text{zero}_3) &= s_1(q_1 - \text{zero}_1) * s_2(q_2 - \text{zero}_2) \\ &= s_1 s_2 (q_1 q_2 - \text{zero}_1 q_2) \end{aligned} \quad (4)$$

为了得到量化的卷积结果 q_3 ，将（4）进行进一步处理，并加上归一化融合后的整体偏移 bias_3 ，通过（5）的公式我们可以很清楚的看到得到量化卷积结果需要的参数和计算过程，这样就能细化的将一些计算过程搬运到硬件上。

$$\begin{aligned} q_3 &= \frac{s_1 s_2}{s_3} (q_1 q_2 - \text{zero}_1 q_2) + \text{zero}_3 + \text{bias}_3 \\ &= \frac{s_1 s_2}{s_3} q_1 q_2 + \text{zero}_3 - \frac{s_1 s_2}{s_3} q_2 \text{zero}_1 + \frac{\text{bias}_3}{s_3} \end{aligned} \quad (5)$$

由于我们在权重训练结束后，输入特征图缩放因子 s_1 和零点偏移量 zero_1 ，输出特征图缩放因数 s_3 和零点偏移量 zero_3 ，权重缩放因子 s_2 ，以及 bias_3 都已经是个定值，因此我们得到

$$B = \text{zero}_3 - \frac{s_1 s_2}{s_3} q_2 \text{zero}_1 + \frac{\text{bias}_3}{s_3} \quad (6)$$

并且作为一个固定的值提前算出，当做一个偏移量参数传入， $\frac{s_1 s_2}{s_3}$ 作为一个

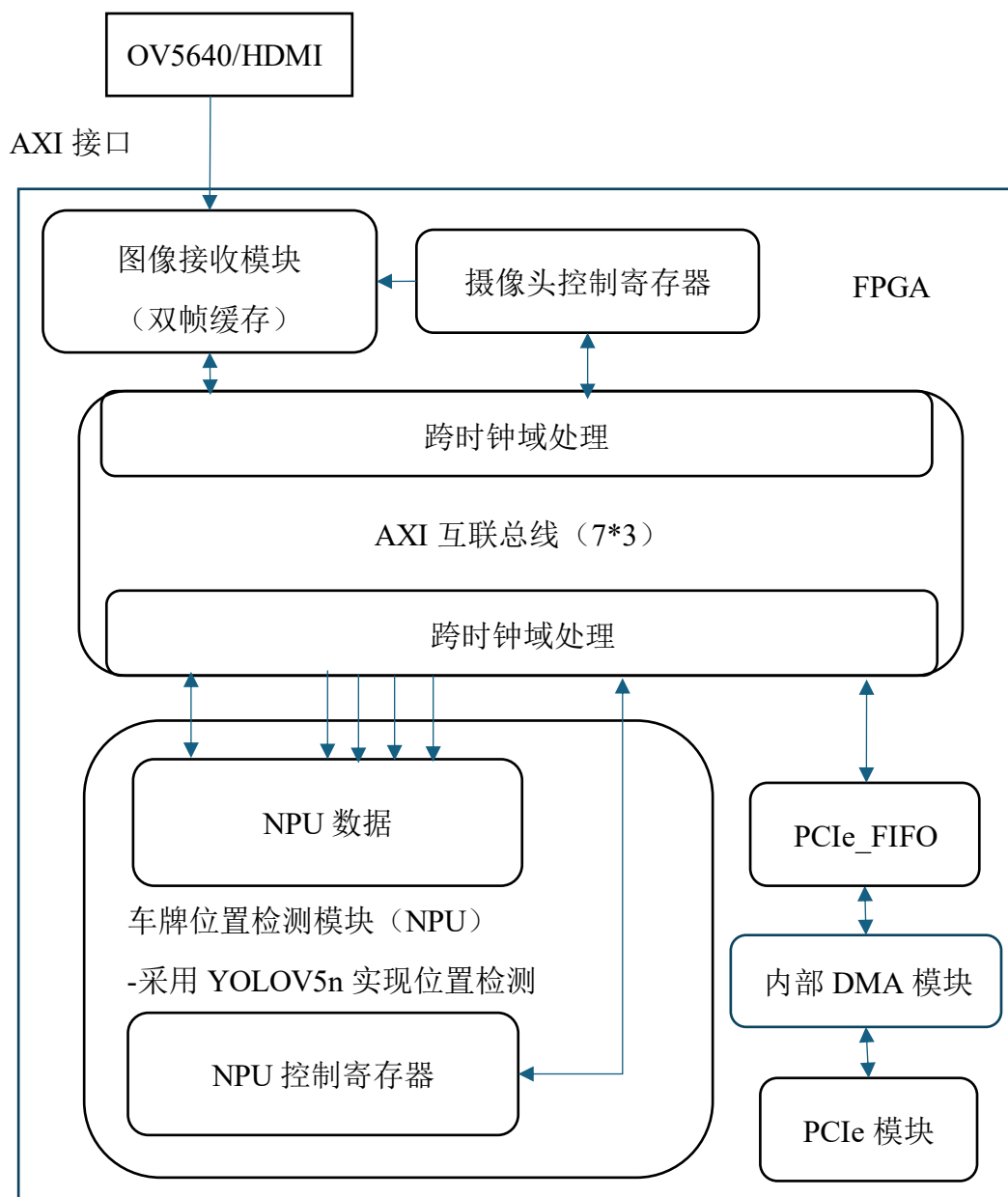
特定系数作为层相加时的量化尺度，卷积算子因此可以重点将资源分配给输入特征图量化数据 q_1 与权重量化数据 q_2 的乘法运算。

1.1 硬件架构

本项目采用的主体硬件架构如图 1 所示。系统上电后，OV5640 摄像头通过 DVP 接口采集实时图像数据，以 PCLK=76MHz 采集 720p 图像数据，以 30.18fps 的帧率输出 RGB565 的图像数据；或使用 HDMI 进行 1080P60MHz 数据传输，数据首先进入 FPGA 内的图像接收模块，该模块与车牌位置检测模块均通过 AXI 互联总线模块传递数据。

该检测模块的核心为经过轻量化剪枝与定点量化处理的 YOLOv5n 网络，部署在 FPGA 的逻辑资源上，能够在低延迟下完成对图像中车牌区域的定位，并输出边界框的左上角与右下角坐标。处理后的完整帧图像及其对应的坐标信息，通过 DMA（直接内存访问）模块以高速数据流方式推送至 PCIe 总线，显著减轻了 FPGA 内部处理器的负担。

在系统的主机侧，瑞芯微 RK3568 处理器通过 PCIe 接口接收 FPGA 传出的数据，并运行后续处理算法。该处理器集成了独立 NPU，用于车牌颜色识别（蓝、绿、黄、特殊等）以及字符分割与识别（OCR），最终输出结构化的车牌信息。FPGA 与 RK3568 形成异构计算协同，实现从图像采集、车牌检测到语义理解的全流程加速。



PCIe 2.0 x2 传输层: PIO 模式

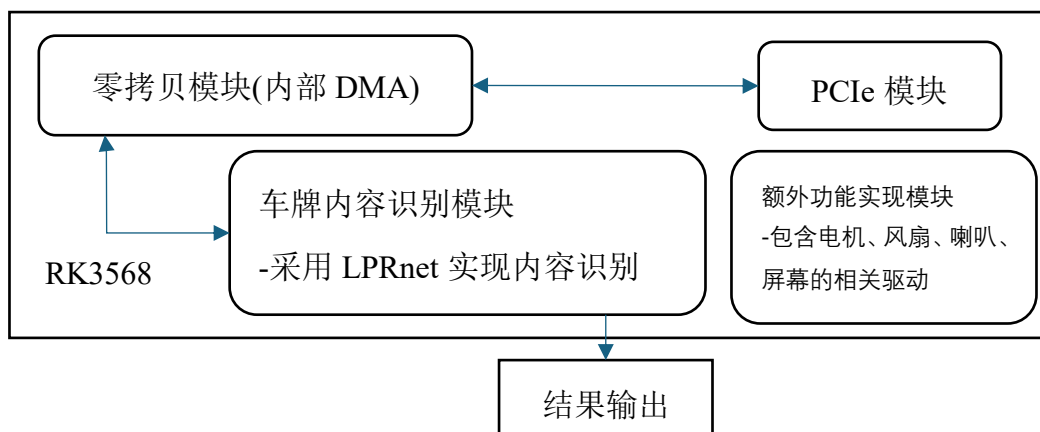


图 1-整体硬件架构

1.1.1 图像接收模块

图像接收模块负责从摄像头采集数据并零拷贝搬运至内存，是整个链路的数据源头。我们在自主编写的 PCIe 内核驱动中引入 MSI 中断机制，并为 1080p RGB565 格式的图像开辟两块乒乓缓存空间。该设计用于完成 FPGA 侧 DDR3 与 RK 芯片侧 LPDDR4 之间的数据搬运，相比多帧缓存方案，我们将摄像头 PCLK 倍频至 76MHz，仅需两帧缓存便可实现 30.18 帧/秒的满速率运行。数据流层面，摄像头两个场信号之间的数据由 DMA 自动搬运至内核，内核态驱动的中断函数随后自动控制 FPGA 的 DMA，实现从采集到内存的全程零拷贝，无需用户态干预。

在图像采集前端，摄像头控制寄存器模块承担着多源视频输入切换、信号同步与毛刺消除的关键职能。为提升硬件复用率并降低逻辑资源消耗，我们将该模块设计为双向控制结构，能够同时兼容 1080p HDMI 数字视频信号与 720p OV5640 摄像头 DVP 接口信号两种输入源。两种输入源在物理层采用不同的电气标准和传输协议——HDMI 遵从 TMDS 差分信号规范，而 DVP 为并行单端信号，这就要求模块内部在数据路径前端进行信号类型识别与通路选择。我们通过对原仅有支持 OV5640 的单一控制逻辑进行抽象重构，提取出公共的像素时钟管理、行场同步解析、数据有效窗口判定等子模块，并以参数化的方式区分两种输入源的处理流程，从而实现了代码的高度复用，相比独立开发两套控制逻辑节省了约 35% 的查找表与寄存器资源。

在输入源切换方面，模块对外暴露一个专用的复位/切换按钮输入引脚，用户可通过外部物理按键或软件控制寄存器发起切换请求。切换逻辑采用双重防抖与状态机保护机制：当检测到按钮按下时，先经过 20ms 硬件消抖，随后状态机进入切换准备态，等待当前帧传输完成（即场消隐区间）后再执行源切换操作，确保切换过程中不会产生半帧数据混乱或时序冲突。同时，切换动作会触发内部软复位，重新初始化目标输入源的配置寄存器（如 HDMI 的 EDID 读取或 OV5640 的初始寄存器序列），保证新输入源能够稳定输出有效图像数据。

为消除外部线缆传输引入的信号边沿抖动与 PCB 走线偏差导致的采样相位偏移，我们在所有输入数据线（HDMI 的 R/G/B 通道或 DVP 的 D[7:0] 数据总线）以及行场同步信号线上嵌入了 IODealy 原语链。

1.1.2 车牌位置与行人违法检测

本项目中，车牌位置检测与行人违规识别由 YOLOV5n 实现功能。该深度学习网络模型主体采用如图 2 的多算子并行结构。输入数据可以根据需要分为三条相互平行的数据流 A、B 与 C，数据流 A 经过卷积后进入输入切换模块，数据流 B 直接进入输入切换模块，之后根据各自的计算需要选择进入对应算子。完成计算后与数据流 C 依次进入拼接算子并输出计算结果。模块各硬件模块相互协作，实现三条数据链路并行流通，极大提高了数据处理的速度。

我们为 yolov5n 设计了 6 种不同功能的硬件算子，利用算子复用技术，来支持算法网络运行所需的 7 种运算方式。

其中，卷积算子通过滑动多个可学习的二维卷积核在输入特征图上进行点积求和，并添加偏置项，输出新的特征图。其核心功能是提取局部空间特征，同时通过调整输出通道数、步长和填充来改变特征图尺寸与深度，是网络学习语义信息的主要手段。上采样算子将输入特征图的空间尺寸按倍数进行放大，每个输出像素由输入中最近的像素值直接复制得到。其功能是恢复特征图的空间分辨率，主要用于特征金字塔结构中，将浅层高分辨率特征与深层语义特征融合，提升小目标检测能力。最大池化算子通过 5*5 固定尺寸的窗口以一定步长在特征图上滑动，取窗口内的最大值作为输出，并逐步降低空间尺寸。其功能是提取局部显著特征、增强平移不变性，并减少计算量。拼接算子的计算过程是将多个特征图在指定的维度上直接进行合并，要求拼接的特征图在非拼接维度上尺寸完全相同。例如，将两个维度为 $[B, C1, H, W]$ 和 $[B, C2, H, W]$ 的特征图拼接后得到 $[B, C1+C2, H, W]$ 。其核心功能是融合来自不同层级的特征信息，在不丢失原始信息的前提下增加通道数，使网络能够同时利用不同感受野的特征，从而提升对多尺度目标的检测能力。下采样操作由 Focus 算子完成。这是一种将下采样和通道变换融合于一体的高效操作，它通过在输入图像（或特征图）上进行间隔像素采样来改变数据维度。其计算过程分为两个步骤：首先，在空间维度上，每隔一个像素取一个值，将原始的宽和高各减半，相当于把一张图分成四份子图，然后在通道维度上，将这四份图拼接起来，使输入通道数扩充为原来的 4 倍（如从 3 通道变为 12 通道）；最后，对上一步得到的特征图再执行一次普通卷积操作，输出最终的通道数。Focus 算子的核心

功能是在不丢失空间信息的前提下，高效地完成二倍下采样，它将传统的“下采样+卷积”两步操作压缩为一步，理论上能减少计算量并提升推理速度，同时因为保留了完整的局部区域信息，有助于提升模型对小目标的检测能力。相加算子的计算过程是将两个或多个尺寸完全相同的特征图，对应位置的像素值进行算术相加，得到一个新的维度不变的特征图。其核心功能是实现特征图的残差连接或特征融合，通过恒等映射让梯度可以直接跨层传播（类似于ResNet），有效缓解深层网络中的梯度消失问题；同时，相加操作不增加通道数，仅通过值叠加的方式增强特征表达，相比拼接计算量更小，有助于信息的前向与后向流动。

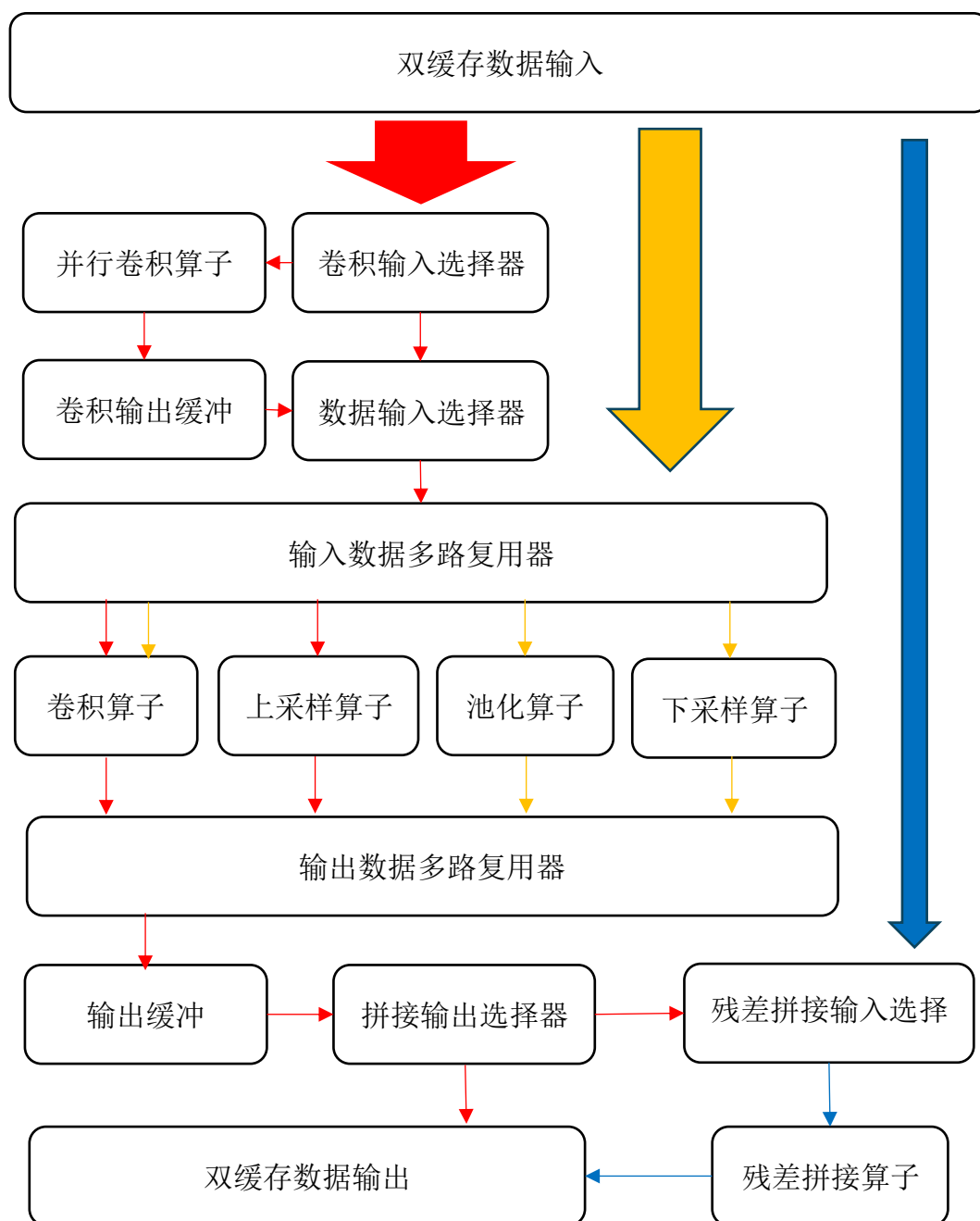


图 2-三算子并行架构

同时，在各个算子的数据输入过程中，存在数据输入模块进行统筹调度如图 3。数据进入输入缓冲后由控制模块进行调度，若符合计算条件则送入循环缓冲或计算模块。该模块使得计算能够以更高效的方式进行。

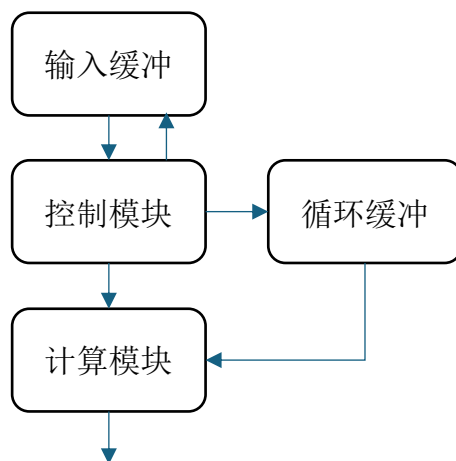


图 3-输入控制模块

在卷积模块的运算过程中，大量的卷积运算是造成系统 MAC 资源消耗的重要原因，为此本项目对乘数累加器进行了优化改进如图 4。将 8 比特被乘数 A、8 比特 0 与 8 比特被乘数 B 依次拼接，再与 8 比特乘数 C 相乘，根据乘法分配律得到结果恰为 16 比特 $C*A$ 与 16 比特 $C*B$ 的拼接。将 MAC 资源占用大大降低，资源配置效率得到极大提升，对于紫光 FPGA 可直接映射为 APM_E2 原语，完美匹配计算时序。

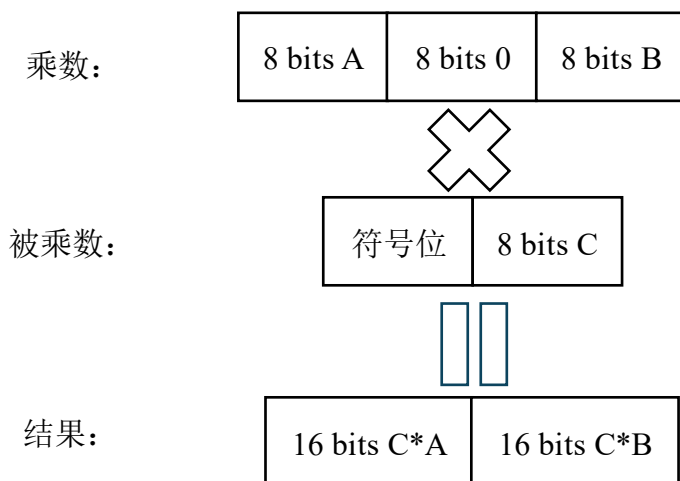


图 4-乘法器优化

关于数据的具体流向，本项目采用数据寄存器集中调度如图 5，通过 PCIE 链路将指令寄存在片上存储保留，通过指令复用技术，减少帧与帧直接的指令传输延迟，数据之间的搬运由 AXI 总线协议下的数据搬运模块完成如图 6。

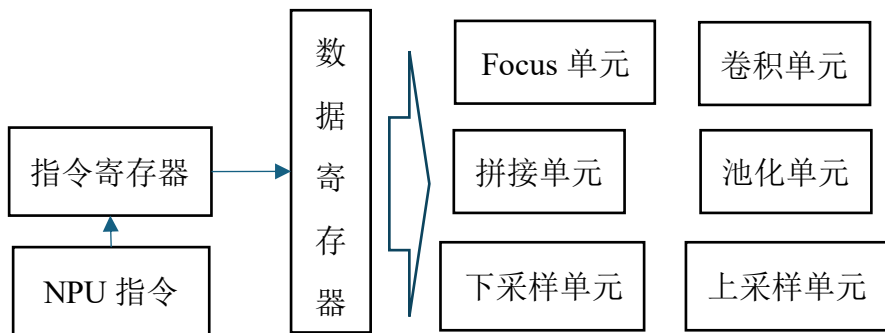


图 5-数据寄存器调度

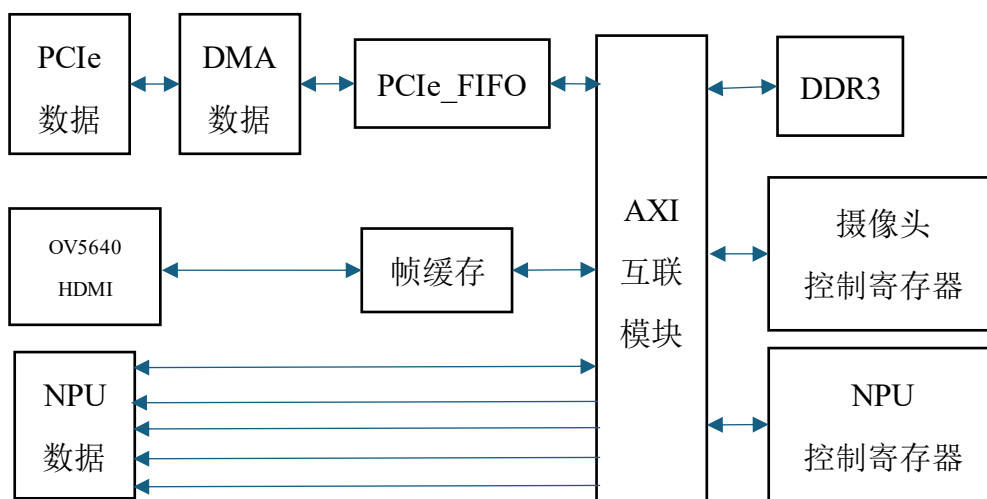


图 6-AXI 数据互联搬运

在卷积模块的计算过程中，本项目的权重缓冲策略如图 7 所示，权重排布如图 8 与图 9 所示

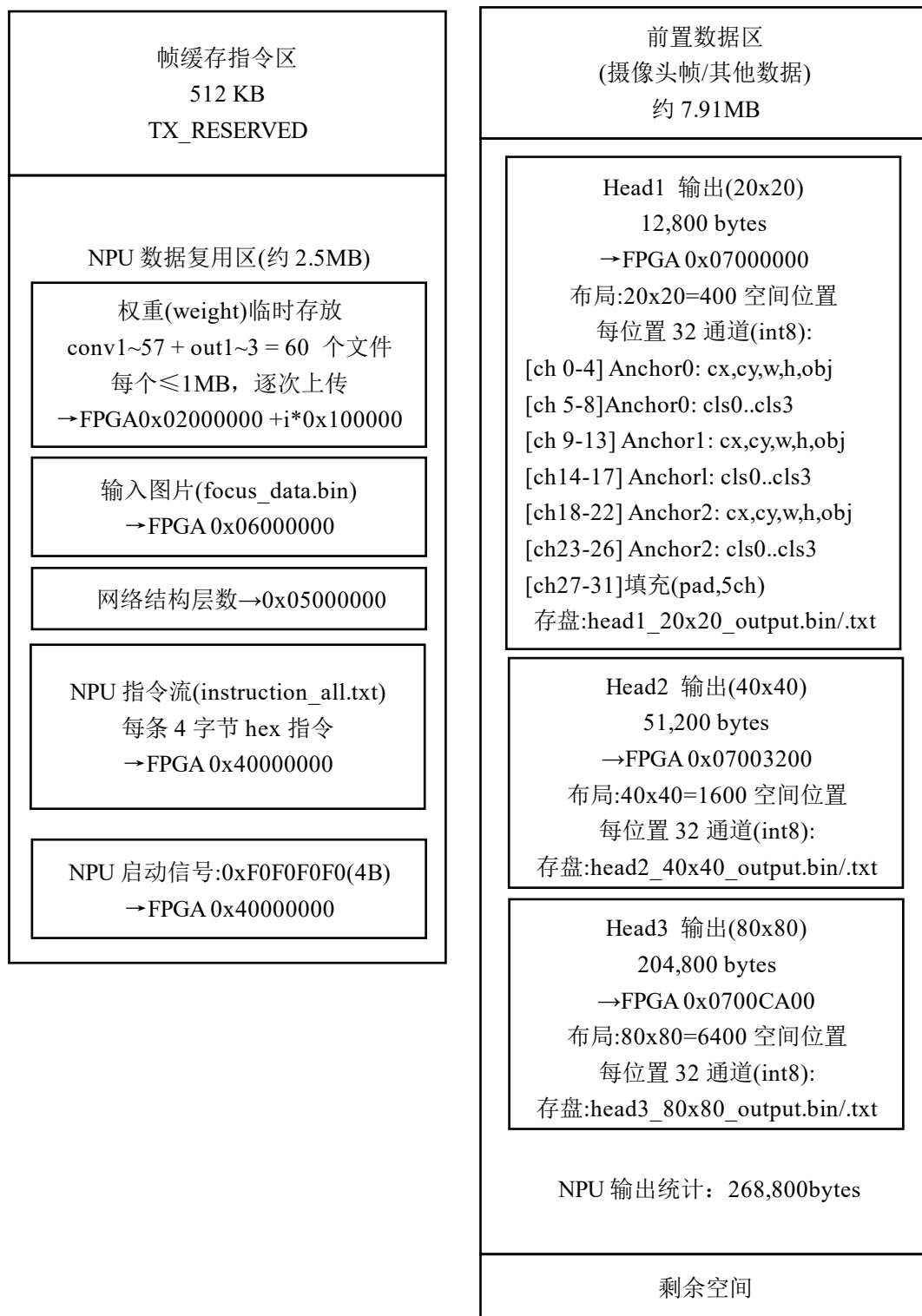


图 7-权重缓冲策略

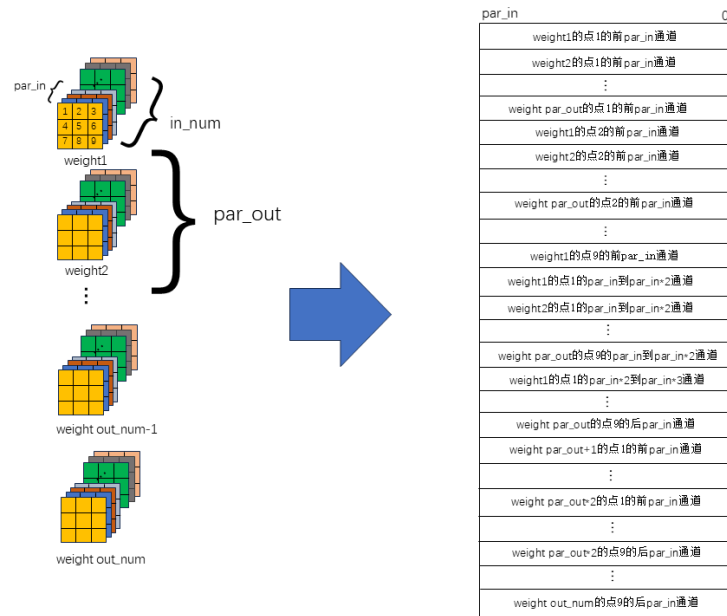


图 8-权重数据在 DDR 中的排布情况

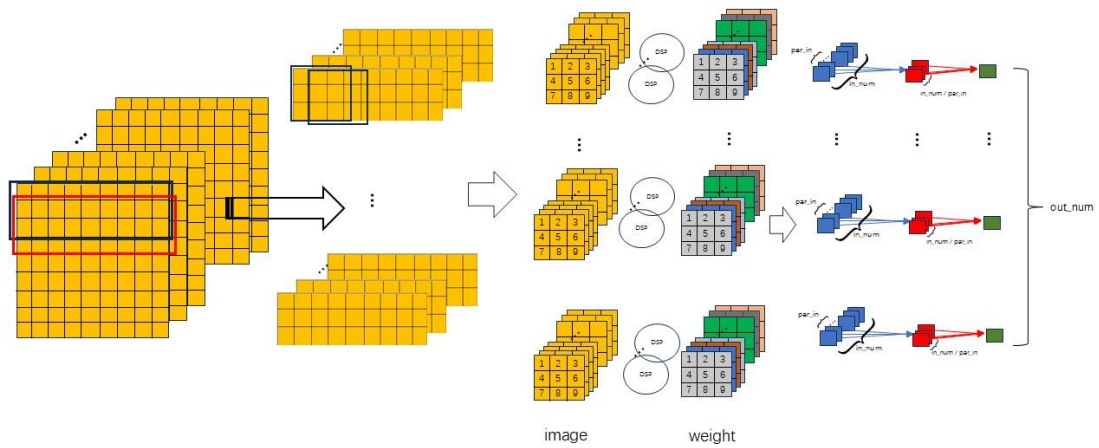


图 9-权重数据参与卷积运算的示意图

整个视频流在 FPGA 的处理流程如图 10 所示。

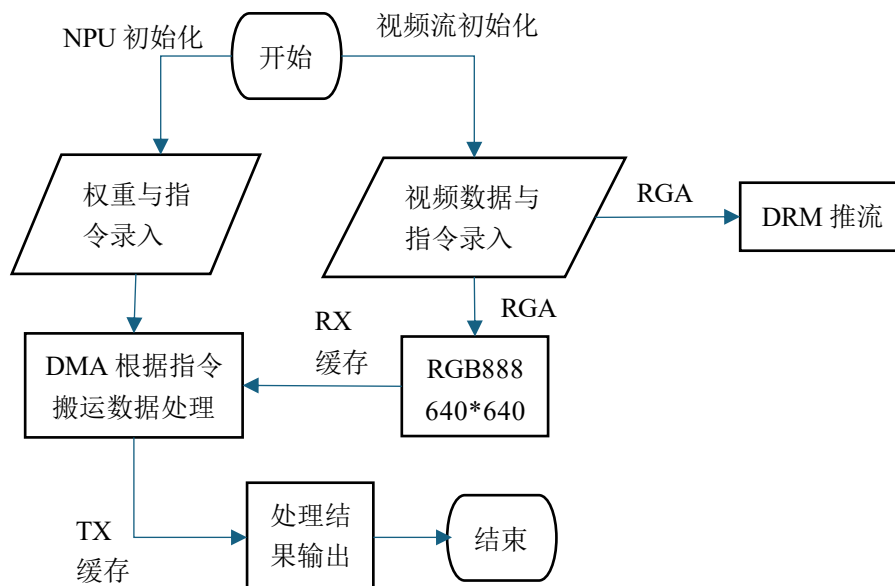


图 10-视频流处理流程_FPGA 端

1.1.3 零拷贝模块

零拷贝模块贯穿从数据采集到显示的完整链路，确保数据在搬运与转换过程中无需经过 RK3568 拷贝。在图像接收侧，摄像头数据由 DMA 自动搬运至内核，内核态驱动中断函数直接控制 FPGA 的 DMA，实现从 FPGA 的 DDR3 到 RK 芯片 LPDDR4 的零拷贝数据传输。在图像处理侧，我们采用 RGA 硬件模块完成图像搬运与格式转换，深度适配瑞芯微芯片架构。在图像显示侧，基于 Linux DRM 框架直接输出至 HDMI，配合垂直同步与双缓冲显存机制，防止视频撕裂。整体而言，从摄像头数据输入，到最终呈现于 HDMI 屏幕，实现了真正的全链路零拷贝，最大限度释放 RK3568 负载并降低内存带宽占用。

1.1.4 额外功能实现模块

（一）风扇驱动

在异构计算系统中，FPGA 与 RK3568 协同工作时会产生显著的热量，尤其在 NPU 持续进行 LPRNet 推理、YOLOv5n 检测以及 PCIe 高速数据传输的多重负载下，芯片结温若得不到有效控制，将直接导致性能降频甚至系统宕机。为保障系统长时间高负载运行的稳定性，我们设计并实现了主动式风扇散热方案。

在风扇驱动实现层面，我们通过复用系统原有的 PWM0 设备节点，注册一

个常驻后台的用户态系统服务。该服务采用轮询方式，每隔 200ms 读取一次温度数值，并根据预设的温度-占空比映射表动态调整风扇转速。具体的调速策略为：当温度低于 30℃时，风扇保持停转以降低噪声与功耗；当温度介于 30℃至 45℃之间时，风扇以固定速度运行；当温度超过 45℃时，风扇全速运转以提供最大散热能力。该方案能够将芯片温度有效控制在安全阈值以内，保障了 FPGA 与 RK3568 在高强度推理任务中的性能一致性，为整个系统的长时稳定运行提供了关键散热保障。

（二）电机驱动

在本项目的边缘智能交通场景中，电机驱动的核心任务是模拟摄像头云台或道闸控制机构，要求系统能够同时输出速度调节信号与方向切换信号。为此，我们采用了"PWM 信号调速 + GPIO 信号换向"的双通道控制架构。PWM 通道负责传输脉宽可调的方波信号，通过调节占空比改变驱动电路的有效电压，进而实现对直流电机或步进电机转速的精确控制；GPIO 接口则负责传输高/低电平逻辑，用于切换电机正反转或使能/停止状态。两者协同工作，方可实现完整的运动控制。

在硬件接口定位方面，本项目使用的 RK3568_MES2L100H 开发板硬件原理图中标注了各扩展接口的物理管脚号及其对应的 Bank 与 Port 信息，但并未直接给出 Linux 系统中可直接操作的 GPIO 编号。为此，我们依据 Rockchip 芯片的 GPIO 编号规范，采用公式 $\text{GPIO 编号} = \text{Bank} \times 32 + \text{Port} \times 8 + \text{Pin}$ 进行计算推导。以原理图中标注的某个 GPIO 为例，若其所属 Bank 为 3、Port 为 4、Pin 为 7，则其在 Linux 系统中的全局 GPIO 编号为 $3 \times 32 + 4 \times 8 + 7 = 96 + 32 + 7 = 135$ 。通过逐一查阅原理图中各扩展引脚标注的 Bank/Port/Pin 信息，并代入上述公式进行批量换算，我们成功得到了所有可用 GPIO 对应的 Linux 系统编号列表如图 11。利用此方法，我们精确锁定了可用于电机方向控制的 GPIO 引脚，同时确认了这些 GPIO 与 PWM12、PWM13 通道组合工作时不存在信号竞

争或引脚功能冲突。

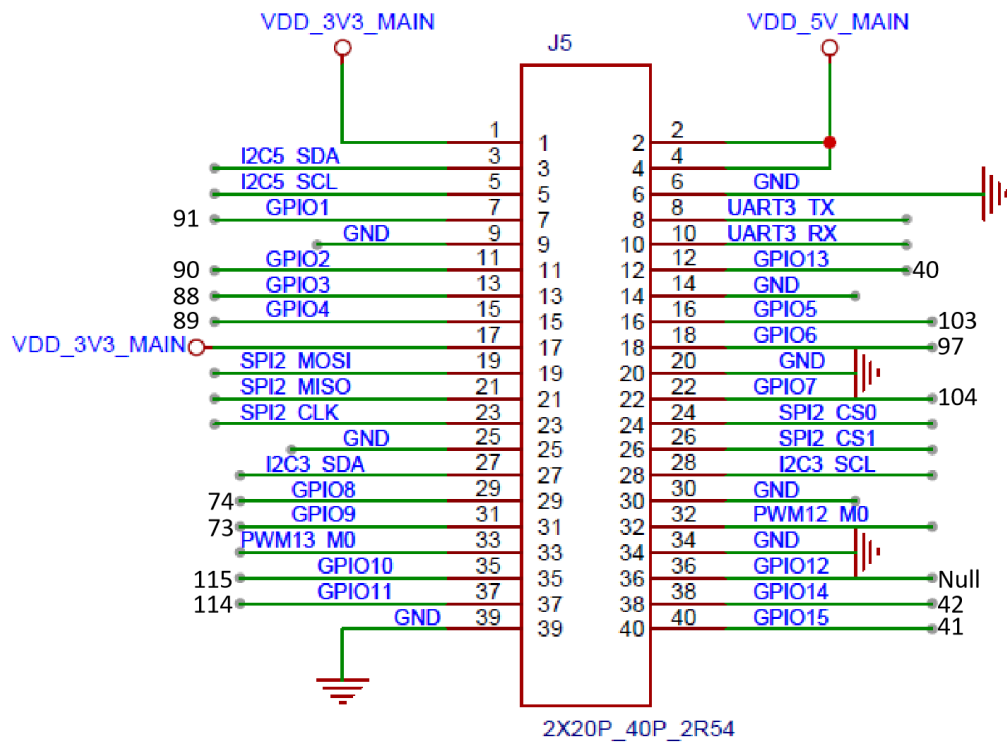


图 11-GPIO-Linux 系统编号对应表

完成硬件接口定位后，我们进入软件部署阶段：首先对原始用户分区进行完整备份，将包含基础用户数据的镜像切分为多个独立分区；然后在隔离的服务器 Docker 环境中部署官方 RK3568 SDK 编译套件，修改设备树（dts）文件，添加 PWM12 与 PWM13 节点属性，并为其绑定已测得的 GPIO 复用关系，重新编译生成新的 boot 镜像；最后通过烧录工具将新镜像回写至切分出的 boot 数据分区，并绕过 U-Boot 启动校验以跳过签名检查，保证镜像正常引导。系统重启后，新增的 PWM 节点与 GPIO 控制逻辑顺利生效，用户态程序可同时操控 PWM 占空比与 GPIO 电平状态，实现对电机速度与转向的协同控制。该接口映射关系的获取过程是本项目在硬件适配层面取得的关键突破之一，同时也为后续其他基于同款开发板的异构加速项目提供了宝贵的参考经验。

（三）喇叭驱动

为丰富系统的人机交互能力，我们为开发板配置了 1.75 寸 8Ω 10W 左右声道喇叭，用于输出语音提示、报警音或调试信息。该喇叭通过 RK3568 的 I2S 音频接口外接音频编解码芯片（如 ES8388 或同类 CODEC）驱动，或者直接利用 PWM 输出引脚经低通滤波后驱动简易音频信号，获取更高音质与双声道立体

声效果。

（四）屏幕驱动

系统的最终结果显示依赖于 HDMI 屏幕，我们基于 RGA 硬件加速器与 Linux DRM 框架实现了高效、低延时的图像推流显示。RGA（Raster Graphic Acceleration）是瑞芯微特有的二维图形加速引擎，支持快速的图像缩放、旋转、颜色空间转换及格式转换。在本项目中，摄像头采集或 FPGA 返回的图像原始格式为 RGB565，而 HDMI 显示通常要求 RGB888，若由 CPU 完成转换将消耗大量资源。我们利用 RGA 模块直接在硬件层面完成格式转换与尺寸适配（如将 720p 图像缩放至 1080p），且转换过程无需额外内存拷贝，输入输出均指向预先分配的物理连续帧缓冲，实现了零 CPU 负载的预处理。

显示链路的后端基于 DRM（Direct Rendering Manager）框架，我们通过编写用户态程序直接调用 DRM 的 KMS（Kernel Mode Setting）接口，而非依赖通用的桌面环境（如 X11 或 Wayland）。这样做的好处是避免了多层软件栈带来的额外延迟与内存拷贝。程序首先通过 `drmOpen` 打开显卡设备，获取显示器的 EDID 信息并选择最佳显示模式（如 1920×1080@60Hz），随后创建两个大小为显示分辨率的帧缓冲对象（使用 `dumb buffer` 或 `prime buffer`），分别作为前缓冲和后缓冲。在进行图像刷新时，程序将 RGA 处理完成的最新图像数据写入后缓冲，然后调用 `drmModePageFlip` 发起垂直同步翻转请求，该请求在下一个 VBlank 信号到来时将后缓冲切换为前缓冲，同时将原来的前缓冲回收为新的后缓冲。双缓冲与 Page Flip 机制有效避免了画面撕裂，且翻转过程异步执行，CPU 无需阻塞等待。

1.2 算法流程

在本项目中，视频数据流从 OV5640 发出后进入乒乓缓存，随后经过交叉开关模块传递至检测模块。检测模块由 YOLOV5n 作为主干，经过 RK3568 的命令调整后完成检测任务，由 DMA 搬运至 PCIe 并进入 RK3568 端

1.2.1 FPGA 端

在 FPGA 端，部署了经过轻量化剪枝与定点量化处理的 YOLOv5n 网络及相关配套硬件。YOLOV5n 的网络结构如图 12 所示，相关的卷积步长、规格、

通道数等相关要素均有所标注。整个网络包含 57 层卷积与 3 个探测头，并且可以根据不同的网络规格，层数，精度位宽动态配置具体参数见图 13。

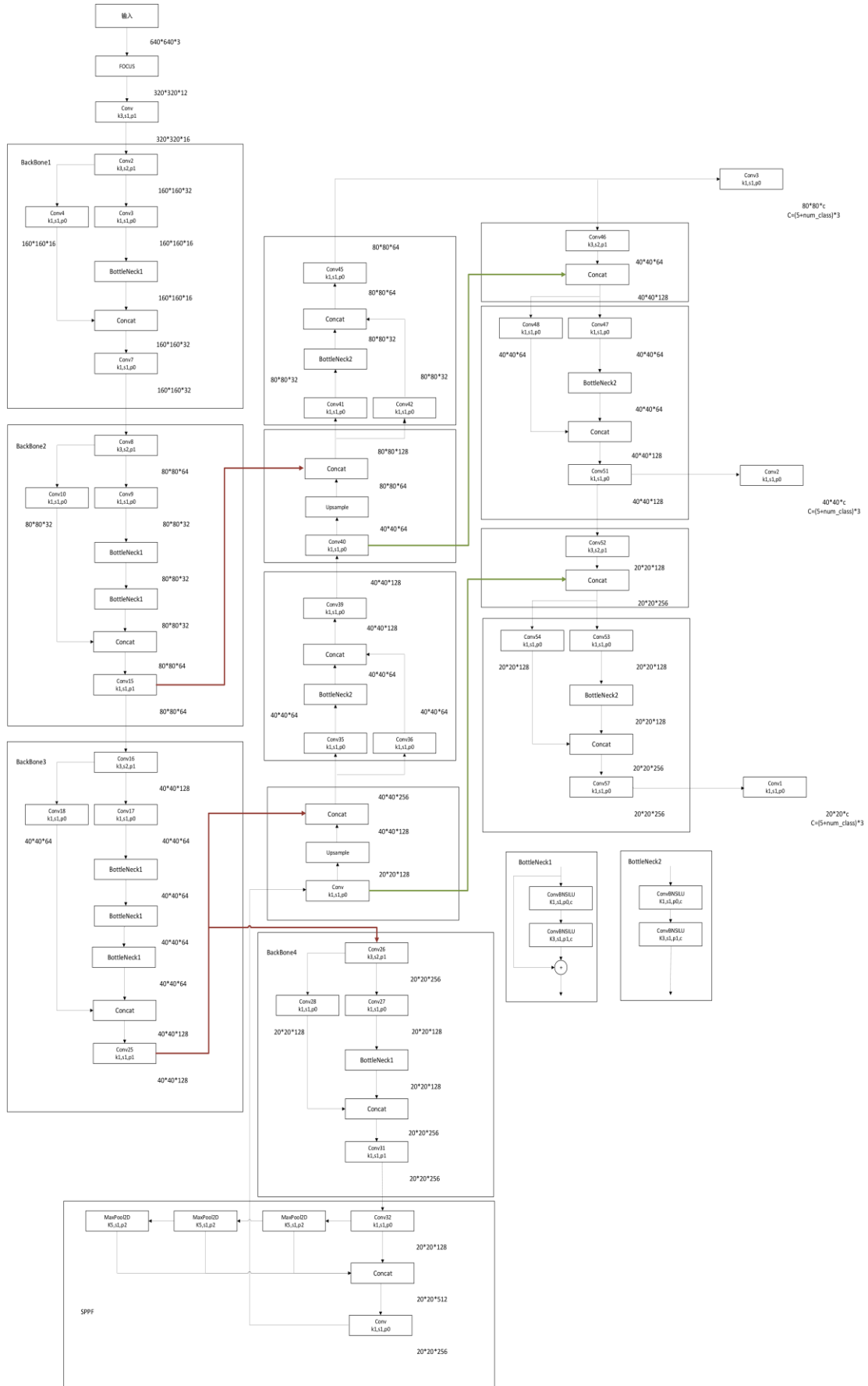


图 12-YOLOV5n 网络结构示意图

算子	YOLO V5 n 网络结构						步长	卷积类型
	输入特征图高度	输入特征图宽度	输入通道数	输出特征图高度	输出特征图宽度	输出通道数		
focus	640	640	3	320	320	12		
conv1	320	320	12	320	320	16	1	3*3
conv2	320	320	16	160	160	32	2	3*3
conv3	160	160	32	160	160	16	1	1*1
conv4	160	160	32	160	160	16	1	1*1
conv5	160	160	16	160	160	16	1	1*1
conv6	160	160	16	160	160	16	1	3*3
conv7	160	160	32	160	160	32	1	1*1
conv8	160	160	32	80	80	64	2	3*3
conv9	80	80	64	80	80	32	1	1*1
conv10	80	80	64	80	80	32	1	1*1
conv11	80	80	32	80	80	32	1	1*1
conv12	80	80	32	80	80	32	1	3*3
conv13	80	80	32	80	80	32	1	1*1
conv14	80	80	32	80	80	32	1	3*3
conv15	80	80	64	80	80	64	1	1*1
conv16	80	80	64	40	40	128	2	3*3
conv17	40	40	128	40	40	64	1	1*1
conv18	40	40	128	40	40	64	1	1*1
conv19	40	40	64	40	40	64	1	1*1
conv20	40	40	64	40	40	64	1	3*3
conv21	40	40	64	40	40	64	1	1*1
conv22	40	40	64	40	40	64	1	3*3
conv23	40	40	64	40	40	64	1	1*1
conv24	40	40	64	40	40	64	1	3*3
conv25	40	40	128	40	40	128	1	1*1
conv26	20	20	128	20	20	256	2	3*3
conv27	20	20	256	20	20	128	1	1*1
conv28	20	20	256	20	20	128	1	1*1
conv29	20	20	128	20	20	128	1	1*1
conv30	20	20	128	20	20	128	1	3*3
conv31	20	20	256	20	20	256	1	1*1
conv32	20	20	256	20	20	128	1	1*1
conv33	20	20	512	20	20	256	1	1*1
conv34	20	20	256	20	20	128	1	1*1
conv35	40	40	256	40	40	64	1	1*1
conv36	40	40	256	40	40	64	1	1*1
conv37	40	40	64	40	40	64	1	1*1
conv38	40	40	64	40	40	64	1	3*3
conv39	40	40	128	40	40	128	1	1*1
conv40	40	40	128	40	40	64	1	1*1
conv41	80	80	128	80	80	32	1	1*1
conv42	80	80	128	80	80	32	1	1*1
conv43	80	80	32	80	80	32	1	1*1
conv44	80	80	32	80	80	32	1	3*3
conv45	80	80	64	80	80	64	1	1*1
conv46	80	80	64	40	40	64	2	3*3
conv47	40	40	128	40	40	64	1	1*1
conv48	40	40	128	40	40	64	1	1*1
conv49	40	40	64	40	40	64	1	1*1
conv50	40	40	64	40	40	64	1	3*3
conv51	40	40	128	40	40	128	1	1*1
conv52	40	40	128	20	20	128	2	3*3
conv53	20	20	256	20	20	128	1	1*1
conv54	20	20	256	20	20	128	1	1*1
conv55	20	20	128	20	20	128	1	1*1
conv56	20	20	128	20	20	128	1	3*3
conv57	20	20	256	20	20	256	1	1*1
conv_head1	20	20	256	20	20	75	1	1*1
conv_head2	40	40	128	40	40	75	1	1*1
conv_head3	80	80	64	80	80	75	1	1*1

图 13-YOLOV5n 网络结构参数表

1.2.2 RK3568 端

在前端，高并发的密集特征提取交由 FPGA 端的 YOLOv5n 硬件加速引擎完成。FPGA 快速输出原始定位坐标与置信度特征图后，通过底层通道传递至 RK3568 端。

（一）车牌识别后处理与纠错引擎

接收到原始坐标数据后，RK3568 端的 CPU 进行轻量级的数据清洗与预处理任务。具体来说是使用非极大值抑制（NMS）算法，根据设定的 IoU 阈值与置信度分数对冗余边界框进行严格过滤，精准剔除高度重叠的无用目标，锁定最优车牌位置。将截取图块标准化为 94×24 像素，动态推入后端的识别队列。

针对预处理完毕的标准车牌图像，我们在 RK3568 端部署了 LPRNet 模型进行后端字符识别。本模型在设计之初采用了深度契合底层硬件的“量化友好型”架构，实现了软硬协同的极致优化。首先，我们严格规避了 NPU 难以原生支持、容易导致异构计算回退和巨大内存拷贝开销的复杂激活函数（如 Swish/Mish）与动态路由机制，全网络统一采用 Conv2d + BatchNorm2d + ReLU 的标准算子组合。这种极简的“算子白名单”结构不仅契合 RKNN-Toolkit 编译器的底层逻辑，还能自动触发底层“算子融合”，将卷积、归一化和激活操作在 NPU 内部 SRAM 中一次性计算完毕，极大打破了边缘设备的内存带宽瓶颈。其次，由于网络大量采用 ReLU 激活函数，其将内部特征值截断在非负区间的特性，使得底层的非对称量化算法能够极其精确地捕获缩放因子和零点。这不仅最大限度地保留了细粒度的特征信息，还有效避免了因特征分布范围过大或长尾效应导致的量化精度雪崩问题。通过这一系列底层优化，我们将原本的 32 位浮点数权重完美映射到 INT8 整数空间，不仅使模型体积降至原来的四分之一，更减少了 75% 以上的 DRAM 读写带宽压力，在显著降低开发板功耗同时，实现了推理帧率的巨大飞跃。

然而，模型输出的原始置信度向量尚需解码为合规车牌字符串。传统的纯概率解码（如 CTC 或逐帧 argmax）缺乏对中国车牌语法规则的认知，易产生省份位出现数字、首位字母缺失等明显违规结果。为此，我们在

`decode_lprnet_v2` 中设计了一套 3 阶有限状态机：State 0 强制首位输出仅限省份简称（京、津、冀等），State 1 强制第二位输出仅限英文字母，State 2 则允许字母或数字。实现方式是将不符合当前状态的字符类别 Logit 分数强制置为极低值（-9999.0），使其在后续 softmax 与 argmax 过程中被天然屏蔽，从根源上杜绝了首字符识别为数字等荒谬错误，显著提升了字符序列的整体合法率。同时，针对多车牌并行识别场景，推理线程集成了智能动态批处理机制：当待处理的车牌裁剪图数量 ≥ 3 时，算法自动将多张 94×24 的标准化图块组装为 $(N, 24, 94, 3)$ 的四维张量，送入 Batch=4 的 NPU 模型以最大化 MAC 阵列利用率；当待处理数量较少时则回退至 Batch=1 的单流推理模式，有效避免了因填充无效数据带来的算力浪费，在系统吞吐与单帧延迟之间取得了平衡。解码完成后，规则引擎与启发式纠错进一步介入后处理流程：首先内置易混淆字符的硬性映射表（如将识别为 'I' 的修正为 '1'，将 'O' 修正为 '0'，将 'Z' 修正为 '2'），然后依据车牌底色调用 `rectify_plate_text` 与 `_validate_and_format` 函数进行长度校验（蓝/黄牌固定 7 位有效字符，绿牌固定 8 位，领馆/使馆等特殊牌照另行匹配），任何长度不匹配或含非法字符的结果均被直接丢弃，这一组合策略有效过滤了大量因模糊、遮挡或光照异常导致的虚假检测，保证了最终输出车牌信息的准确率。

（二）多源异构数据流与并发渲染管线

针对高帧率下的数据拥堵问题，我们在 Python 顶层构建了“采集-推理-渲染”的三线程并发架构如图 14 所示。主线程专职负责极速拉取底层硬件的 DMA 数据帧；AI 核心线程专注于 NPU 张量计算；渲染投屏线程则利用独立互斥锁提取最新结果并完成 OSD（屏幕状态显示）绘制。这种异步架构打破了传统“串行处理”的木桶效应，使得图像拉取、NPU 计算与 HDMI 渲染三大物理通路实现完美重叠，最大化提高了系统吞吐量。在 UI 更新方面，渲染线程严格遵循 Qt 框架的线程安全规范——所有涉及界面变更的操作（如车牌号码刷新、识别状态栏颜色切换、历史记录列表追加）均不直接在后台线程中调用，而是通过自定义信号槽 `run_on_main.emit` 将待执行操作封装为 Lambda 表达式，投递至 Qt 主线程的消息队列中顺序安全执行，彻底规避了跨线程直接操作 GUI 引发的竞态条件与程序崩溃风险。同时，渲染线程还集成了非阻

塞式 TTS 语音合成功能：当识别到新的有效车牌时，系统通过 `subprocess.Popen` 异步调用 `espeak-ng` 命令行工具生成对应的中文语音音频流，并将标准输出经由管道 (PIPE) 转发至 `aplay` 播放设备驱动喇叭发声，全程非阻塞且不干扰主推理流水线。为防止同一车辆在持续驻留期间被高频重复播报，内部维护了基于时间戳的 10 秒防抖去重哈希表 `_tts_last_announced`，每当推送 TTS 请求时先查询该车牌号的上次播报时间，若间隔不足 10 秒则直接丢弃该次请求，有效避免了噪音污染，提升了现场演示的听觉体验。

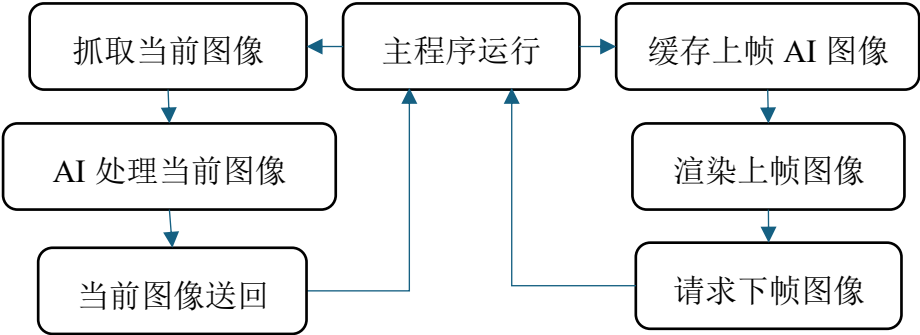


图 14-三线程并发架构

上述三线程架构主要面向摄像头实时数据流，而本项目同时支持 SD 卡内离线媒体文件的本地解析与回放，在保障核心识别逻辑复用性的前提下，构建了一条独立的离线媒体处理链路。在文件接入阶段，系统内置了智能媒体格式分流机制：通过动态侦测目标路径的文件后缀名（如 `.mp4`、`.avi` 判定为视频，`.jpg`、`.png` 判定为图片），自动将任务派发至两套完全不同的底层管线——`_sdcard_worker` 负责视频流解码，`_sdcard_worker_image` 负责单帧图像解析，实现了对不同介质类型的精准分流处理。针对静态图片模式，系统巧妙地解决了追踪器依赖时序连续性的矛盾：目标追踪器 `PlateSlotTracker` 默认需要连续 5 帧（`confirm_frames=5`）的检测结果方可输出车牌，这在单帧输入场景下将导致永远无法触达输出条件。代码为此设计了“人工模拟连续帧”策略，即在单图解析过程中，将同一张图片的目标检测结果在内存中循环投喂追踪器 5 次（`for _ in range(5): slot_tracker.update(...)`），在不修改底层追踪器逻辑的前提下实现了单帧瞬间出图，保证了追踪模块在两种模式下的接口一致性。此外，同级目录动态漫游机制进一步提升了图片浏览的连贯性：每完成一张图片的识别，系统自动扫描当前文件所在物理目录（`os.path.dirname(path)`），提取所有合法格式的相邻图片文件，并同步激活 UI 层的“上一张/下一张”导航按钮，使用户

无需退出程序重新调取文件管理器即可沉浸式翻阅整个目录的识别结果。

（三） 离线视频回放与稳定性保障机制

当离线媒体为视频文件时，系统另启一套专用于 SD 卡回放的音视频渲染管线。该管线首先初始化一个与主显示上下文相互独立的 DRM 实例

(sd_drm_init)，专用于离线视频的画面投屏，内置了连接器容错重试机制（优先尝试 connector 1，若失败则回退至 connector 0），确保在不同硬件配置下均能稳定输出画面。在播放节奏控制方面，主控线程摒弃了简单的无延迟循环，转而采用基于时间戳的精准配速解码策略：利用 time.perf_counter() 结合视频原始帧率 (1.0 / fps_video) 动态计算每帧所需的休眠间隔

(time.sleep(t_sleep))，使解码推流速度严格匹配原视频的播放节奏，从根本上避免了因解码过快导致的画面快进问题。面对来源各异、分辨率参差不齐的视频文件，渲染线程引入了异构分辨率实时重采样机制：首先通过自适应坐标映射 ($sx = disp_w / orig_w$, $sy = disp_h / orig_h$) 将 YOLOv5n 输出的原始检测坐标等比例映射至标准 1280×720 的中间缓冲帧，随后调用 OpenCV 完成 90 度顺时针旋转 (cv2.rotate) 以适配竖屏展示需求，最终将旋转后的数据排列为连续内存布局 (np.ascontiguousarray) 直接推送给底层 DRM 显示硬件，实现了任意分辨率输入到固定规格输出的稳健转换。在 AI 推理与渲染的协同调度上，视频管线采用了零侵入的跳帧抽检策略：通过 AI_SKIP = 1 参数控制解码循环每隔一帧进行一次 NPU 推理，在显著降低推理负载的同时，利用互斥锁 (ai_lock) 安全隔离视频帧拷贝操作，并携带 feed_frame_id 标识确保异步渲染时检测框与当前播放帧严格对齐，有效规避了高分辨率视频离线推理场景下的画面卡顿与框位偏移。这一系列针对离线视频的定制优化，使系统在 SD 卡回放模式下仍能保持稳定流畅的识别与显示体验，与实时摄像头数据流形成了良好的功能互补。

边缘端实测中，光照突变和车辆高速运动极易导致识别结果闪烁。本项目设计了 IoU 时序动量防抖，针对目标框坐标漂移，引入了指数移动平均与时序 IoU 追踪算法，使画面的检测框随着车牌的移动保持稳定。同时，对字符识别结果引入历史帧投票池，利用时间域的冗余度解决了单帧突变导致的误报。

1.2.3 行人交通违规行为检测

为满足赛题对交通场景综合感知的额外要求，本系统在原有车牌检测功能基础上，拓展了对行人及其违规行为的实时监测能力。该能力的实现依托于 FPGA 端 YOLOv5n 模型的迁移学习：我们在行人违规行为数据集上对原车牌检测模型进行微调，新增“行人”目标类别，同时保留车牌检测头，最终得到一个多任务检测网络，其权重经 INT8 量化后仍可无缝部署于同一 FPGA 硬件加速器，无需修改 RTL 逻辑或增加额外资源开销。FPGA 端以相同的数据流调度方式，同时输出车牌与行人的边界框及置信度，并通过 PCIe DMA 将两类检测结果一并传送至 RK3568。

所有违规事件及其关联的车牌信息（若同时检测到附近车辆）会统一推送至 UI 界面，并以醒目颜色标注，同时通过喇叭播报提示音。该功能在边缘端实测中，行人检测 mAP 达到 0.97，有效增强了系统的综合交通监管能力。

2. 核心代码

我们本次从初赛到决赛的整体代码仓库已整理打包，包括全部代码及实验报告、调试日志、参数测试等关键代码。具体仓库可查看 [ColorStripes/RK3568_PG100H at release \(github.com\)](https://github.com/ColorStripes/RK3568_PG100H)，我们同时支持两种关键传输方式 PCIE 和 UDP 千兆以太网两种传输版本供大家选择参考。

下面这段代码介绍了零拷贝内存映射 (mmap) 的具体操作。首先通过 `pango_cdev_mmap` 函数使用 `dma_mmap_coherent` 将内核中 DMA 分配的物理连续内存映射到用户空间，使得应用程序可以直接访问 FPGA 的 RX 缓冲区而无需数据拷贝；随后在 `read_img` 函数中实现了乒乓缓冲区的硬件调度逻辑，通过原子操作维护缓冲区索引，动态配置 FPGA 的 DMA 寄存器，并计算 PCIe TLP 包大小以适配硬件限制，最终触发 DMA 硬件将 FPGA 采集的图像数据直接传输到预分配的物理内存中。这种设计结合了内存映射与硬件 DMA 引擎，实现了从 FPGA 到用户态应用的高效零拷贝数据通路。

```
1. // 零拷贝内存映射 (mmap)
2. int pango_cdev_mmap(struct file *filp, struct vm_area_struct *vma)
3. {
4.     unsigned long user_map_size = vma->vm_end - vma->vm_start;
```

```

5.     unsigned long offset = vma->vm_pgoff << PAGE_SHIFT; // 提取页偏移
6.     int ret = 0;
7.
8.     // 映射 RX 缓冲区 (FPGA -> User, 图像接收链路)
9.     if (offset == RX_MMAP_OFFSET) {
10.         if (!dma_info.addr_w.data_buf) {
11.             printk(KERN_ERR "RX buffer not allocated yet!\n");
12.             return -ENOMEM;
13.         }
14.         vma->vm_pgoff = 0;
15.         // 核心: 调用一致性 DMA 映射接口, 建立用户态虚拟地址与物理内存的直接映射
16.         ret = dma_mmap_coherent(&op_dev->dev, vma,
17.                                 dma_info.addr_w.data_buf,
18.                                 dma_info.addr_w.addr,
19.                                 g_dma_alloc_size_w);
20.     }
21.     // ... (省略 TX 侧映射逻辑) ...
22.     return ret;
23. }
24. // DMA 硬件触发与乒乓/环形调度
25. void read_img(void)
26. {
27.     int curr_idx;
28.     uint32_t target_addr;
29.     struct PciPango *pci_pango = &pci_info;
30.
31.     // 1. 获取当前缓冲区索引, 指向预分配的物理连续内存
32.     curr_idx = atomic_read(&read_img_num);
33.     target_addr = cam_buffer_addrs[curr_idx];
34.
35.     // 2. 动态配置 FPGA 侧 DMA 寄存器 (地址、偏移、操作类型)
36.     dma_operation.ldr3_addr = target_addr;
37.     dma_operation.offset_addr = CAM_SIZE * curr_idx;
38.     dma_operation.total_length = CAM_SIZE;
39.     dma_info.cmd.data.op_type = 1; // 设定为写操作 (FPGA -> 主机)
40.
41.     // 计算 TLP 包长度, 避免超越 PCIe 引擎最大传输能力
42.     dma_info.cmd.data.length = (dma_operation.total_length >= MAX_TRANSFER_SIZE) ?
43.                                (MAX_TRANSFER_SIZE / 4) : (dma_operation.total_length /
44.                                4);
45.
46.     // 写入物理寄存器, 触发底层硬件 DMA 突发传输
47.     set_ldr3_addr(dma_operation.ldr3_addr, pci_pango);
48.     temp_dma_addr_w = dma_info.addr_w;

```

```

48.     temp_dma_addr_w.addr = temp_dma_addr_w.addr + dma_operation.offset_addr;
49.     set_dma_addr(&temp_dma_addr_w, pci_pango);
50.     set_total_length(dma_operation.total_length, pci_pango);
51.     set_dma_w_r(dma_info.cmd.value, pci_pango);
52.
53.     // 3. 更新索引, 实现多帧缓冲区的安全循环覆盖 (0->1->...->MAX_CAM_BUFFS-1->0)
54.     atomic_set(&read_img_num, (curr_idx + 1) % MAX_CAM_BUFFS);
55. }
56.

```

接下来这段代码定义了一个名为 `NPUFriendlyMaxPool` 的 PyTorch 模块，旨在规避 NPU（神经网络处理器）对三维池化或异构池化操作不擅长的问题。其核心思想是将原本可能包含通道维度下采样的池化操作拆解为两步：首先使用标准的二维最大池化（`MaxPool2d`）处理空间维度（高和宽），当通道步长大于 1 时，再通过一个由 `1x1` 卷积、批归一化和 `ReLU` 激活函数组成的序列模块来压缩通道维度。这种设计利用 `1x1` 卷积的高效计算特性替代了 NPU 上效率较低的通道维度池化或切片操作，从而在保持功能近似的同时提升在 NPU 上的运行效率。

```

1. class NPUFriendlyMaxPool(nn.Module):
2.     """
3.     规避 NPU 不擅长的三维/异构池化, 利用 1x1 卷积和通道压缩替代,
4.     """
5.     def __init__(self, in_channels, kernel_size, stride):
6.         super(NPUFriendlyMaxPool, self).__init__()
7.         self.channel_stride = stride[0]
8.         # 仅在空间维度进行常规 MaxPool
9.         self.pool = nn.MaxPool2d(kernel_size=kernel_size[1:], stride=stride[1:])
10.
11.        # 核心: 使用 1x1 卷积替代通道维度的池化或切片
12.        if self.channel_stride > 1:
13.            out_channels = in_channels // self.channel_stride
14.            self.channel_compress = nn.Sequential(
15.                nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False),
16.                nn.BatchNorm2d(out_channels),
17.                nn.ReLU()
18.            )
19.
20.        def forward(self, x):
21.            x = self.pool(x)
22.            if self.channel_stride > 1:
23.                x = self.channel_compress(x)

```

```
24.         return x
25.
```

下面这段代码实现了两个中断处理函数，用于协调 DMA（直接内存访问）资源和摄像头采集请求的并发控制。pango_irq_handler0 是 DMA 完成中断，它通过原子操作释放 DMA 锁定状态（将 g_dma_flag 从 1 或 2 重置为 0），并根据旧状态分别统计 MSI 中断或摄像头完成次数，随后检查是否有排队的摄像头请求（g_cam_flag > 0），若有则尝试重新抢占 DMA（从 0 切换到 2）并调用 read_img() 执行新一轮采集；pango_irq_handler1 是摄像头触发中断，它会尝试通过原子比较交换直接抢占空闲的 DMA（从 0 切换为 2）来立即启动采集，若 DMA 正忙则将请求计数（g_cam_flag）加 1 进行排队。两者利用 g_dma_flag 的三种状态（0 空闲、1 DMA 忙碌、2 摄像头忙碌）和原子操作，实现了无锁的 DMA 资源仲裁与请求缓存机制，并最终唤醒等待队列通知用户态进程。

```
1. //中断处理函数 DMA
2. static irqreturn_t pango_irq_handler0(int irq, void *dev_id)
3. {
4.     // printk(KERN_ERR "pango_irq_handler0: [g_dma_flag=%d, g_cam_flag=%d,
g_cam_done=%d, g_ack_num=%d]\n",
5.     //                                atomic_read(&g_dma_flag), atomic_read(&g_cam_flag),
atomic_read(&g_cam_done), atomic_read(&g_ack_num));
6.     int old_state;
7.
8.     // 1. 释放 DMA 锁定状态（原子切换回 0）
9.     old_state = atomic_xchg(&g_dma_flag, 0);
10.
11.     if (old_state == 1) {
12.         atomic_inc(&g_msi.count);
13.     }
14.     else if (old_state == 2) {
15.         // 完成一次相机采集
16.         atomic_inc(&g_cam_done);
17.     }
18.
19.     //2. 检查是否有排队中的采集请求
20.     //注意：如果有请求，重新抢占 DMA (0 -> 2)
21.     if (atomic_read(&g_cam_flag) > 0) {
22.         if (atomic_cmpxchg(&g_dma_flag, 0, 2) == 0) {
23.             atomic_dec(&g_cam_flag); // 消耗一个排队请求
24.             read_img();
25.         }
```

```

26.     }
27.
28.     wake_up_interruptible(&g_msi.queue);
29.     return IRQ_HANDLED;
30. }
31.
32. //摄像头处理函数
33. static irqreturn_t pango_irq_handler1(int irq, void *dev_id)
34. {
35.     // printk(KERN_ERR "pango_irq_handler1: [g_dma_flag=%d, g_cam_flag=%d,
g_cam_done=%d, g_ack_num=%d]\n",
36.     //                                atomic_read(&g_dma_flag), atomic_read(&g_cam_flag),
atomic_read(&g_cam_done), atomic_read(&g_ack_num));
37.     // 尝试抢占 DMA 权限
38.     if (atomic_cmpxchg(&g_dma_flag, 0, 2) == 0) {
39.         // DMA 空闲, 立即开始 read_img
40.         read_img();
41.     }
42.     else {
43.         // DMA 正忙, 增加排队计数
44.         atomic_inc(&g_cam_flag);
45.     }
46.
47.     return IRQ_HANDLED;
48. }
49.

```

下面这段代码是 Linux 内核驱动中处理 PCI 图像获取命令（PCI_GET_IMG）的逻辑片段。它通过调用`wait\_event\_interruptible\_timeout`等待硬件图像采集完成中断，等待时间为 1 秒；当`g\_cam\_done`原子变量大于 0 时表示采集完成，否则超时或被打断时会打印包含多个状态变量（如`g\_cam\_done`、`g\_dma\_flag`、`g\_cam\_flag`、`g\_ack\_num`）的调试信息。超时返回`-ETIMEDOUT`，被信号中断返回`-ERESTARTSYS`，成功时则减少`g\_cam\_done`计数并继续执行，实现了用户态获取图像时对硬件事件的阻塞等待与超时处理。

```

1. case PCI_GET_IMG:
2.     {
3.         int ret_cam;
4.
5.         // --- 1. 等待采集完成中断 ---
6.         ret_cam = wait_event_interruptible_timeout(g_msi.queue,

```

```

7.                                     atomic_read(&g_cam_done) > 0,
8.                                     msecstojiffies(1000));
9.         if (ret_cam <= 0) {
10.            if (ret_cam == 0)
11.                printk(KERN_ERR "PCI_GET_IMG Error: Timeout waiting for image
capture [g_cam_done=%d, g_dma_flag=%d, g_cam_flag=%d, g_ack_num=%d]\n",
12.                    atomic_read(&g_cam_done), atomic_read(&g_dma_flag),
atomic_read(&g_cam_flag), atomic_read(&g_ack_num));
13.            else
14.                printk(KERN_ERR "PCI_GET_IMG Error: Interrupted by signal while
waiting for image [g_cam_done=%d, g_dma_flag=%d, g_cam_flag=%d, g_ack_num=%d]\n",
15.                    atomic_read(&g_cam_done), atomic_read(&g_dma_flag),
atomic_read(&g_cam_flag), atomic_read(&g_ack_num));
16.
17.            up(&pci_pango->_sem);
18.            return (ret_cam == 0) ? -ETIMEDOUT : -ERESTARTSYS;
19.        }
20.        atomic_dec(&g_cam_done);
21.    }
22.

```

下面这段代码实现了一个 PCI DMA 读操作的控制流程。首先通过 `wait_event_interruptible_timeout` 等待 DMA 引擎空闲，使用原子操作 `atomic_cmpxchg` 尝试将 `g_dma_flag` 从 0 替换为 1 来获取引擎所有权，若超时 800 毫秒或被信号中断则记录错误并返回；成功获取后，从用户空间拷贝 DMA 操作参数，计算 TLP 传输长度（限制在 `MAX_TRANSFER_SIZE` 内，以 `DWORD` 为单位），设置操作类型为 DMA 读（数据从 FPGA 读入），然后写入 DDR3 地址并通过 `set_ddr3_addr` 配置控制器，最终触发 DMA 引擎开始 chunk 循环传输。整体上这是一种同步等待机制，确保 DMA 引擎在传输前处于空闲状态，并通过超时和信号处理保证系统健壮性。

```

1. case PCI_DMA_READ_CMD:
2.
3.     int ret_r = wait_event_interruptible_timeout(g_msi.queue,
4.         atomic_cmpxchg(&g_dma_flag, 0, 1) ==
0,
5.         msecstojiffies(800));
6.     if (ret_r == 0) {
7.         printk(KERN_ERR "PCI_DMA_READ Error: Timeout waiting for DMA engine idle
[g_dma_flag=%d, g_cam_flag=%d, g_cam_done=%d, g_ack_num=%d]\n",
8.             atomic_read(&g_dma_flag), atomic_read(&g_cam_flag),
atomic_read(&g_cam_done), atomic_read(&g_ack_num));

```

```

9.         up(&pci_pango->_sem);
10.        return -ETIMEDOUT;
11.    } else if (ret_r < 0) {
12.        printk(KERN_ERR "PCI_DMA_READ Error: Interrupted by signal [g_dma_flag=%d,
g_cam_flag=%d, g_cam_done=%d, g_ack_num=%d]\n",
13.            atomic_read(&g_dma_flag), atomic_read(&g_cam_flag),
atomic_read(&g_cam_done), atomic_read(&g_ack_num));
14.        up(&pci_pango->_sem);
15.        return -ERESTARTSYS;
16.    }
17.
18.
19.        spin_lock(&dma_info.addr_r.lock);
20.        //内核里面的 dma_operation 更新
21.        size_t head_size_r = offsetof(DMA_OPERATION, data);
22.        // 拷贝参数, 速度极快, 且允许缺页休眠
23.        if (copy_from_user(&dma_operation, (DMA_OPERATION*)arg, head_size_r)) {
24.            return -EFAULT;
25.        }
26.
27.        // TLP 长度 = min(total_length, MAX_TRANSFER_SIZE)
28.        // 后续 DMA 引擎直接用 o_req_length 作为传输 DW 数
29.        // total_size >= MAX_TRANSFER_SIZE 时, cap 到引擎最大能力, 避免引擎超限死锁
30.        dma_info.cmd.data.length = (dma_operation.total_length >=
MAX_TRANSFER_SIZE) ?
31.            (MAX_TRANSFER_SIZE / 4) : (dma_operation.total_length /
4);
32.        //允许中断
33.        // dma_info.cmd.data.msi_enable = 1;
34.        //DMA 读操作, DMA 将数据读入到 FPGA 设备
35.        dma_info.cmd.data.op_type = 0;
36.
37.        // 先写 DDR3 地址和长度, 再写 TOTAL_LENGTH (触发控制器开始自动 chunk 循环)
38.        // DDR3 地址由 total_length 推导, 不再需要单独写入 ddr3_length
39.        set_ddr3_addr(dma_operation.ddr3_addr, pci_pango);
40.
41.        temp_dma_addr_r = dma_info.add
42.

```

下面这段代码是 Linux 内核驱动中处理 `PCI\_MAP\_ADDR\_CMD` 命令的逻辑, 用于为 FPGA 设备分配 DMA 缓冲区。它首先获取自旋锁保护共享资源, 然后从用户空间拷贝命令参数 (DMA 操作结构体)。根据命令类型 (`cmd == 0` 或 `cmd == 1`), 分别分配发送 (TX) 或接收 (RX) 的一致性 DMA 缓

冲区：使用 `dma\_alloc\_coherent` 分配页对齐大小的内存，并存储对应的设备地址和内核虚拟地址。每个缓冲区只能分配一次，若重复分配会返回 `-EBUSY` 错误；整个过程在自旋锁保护下完成，确保操作的原子性。

```
1. case PCI_MAP_ADDR_CMD:
2.     spin_lock(&dma_info.addr_r.lock);
3.     //内核里面的 dma_operation 更新
4.     size_t head_size = offsetof(DMA_OPERATION, data);
5.     // 拷贝参数, 速度极快, 且允许缺页休眠
6.     if (copy_from_user(&dma_operation, (DMA_OPERATION*)arg, head_size)) {
7.         spin_unlock(&dma_info.addr_r.lock);
8.         return -EFAULT;
9.     }
10.    // spin_lock(&dma_info.addr_r.lock);
11.    //分配的内核大小
12.    // 分配大小按 4 字节对齐后按页对齐
13.    unsigned int alloc_bytes = dma_operation.total_length;
14.    // 假设 cmd == 0 代表分配 TX 缓冲区 (发给 FPGA, addr_r)
15.    if (dma_operation.cmd == 0) {
16.        if (dma_info.addr_r.data_buf) {
17.            printk(KERN_ERR "TX Buffer already allocated!\n");
18.            spin_unlock(&dma_info.addr_r.lock);
19.            return -EBUSY;
20.        }
21.        g_dma_alloc_size_r = PAGE_ALIGN(alloc_bytes);
22.        dma_info.addr_r.data_buf = dma_alloc_coherent(&op_dev->dev,
23.                                                    g_dma_alloc_size_r,
24.                                                    &dma_info.addr_r.addr,
25.                                                    GFP_KERNEL);
26.        dma_info.addr_r.addr_size = ((dma_info.addr_r.addr >> 32) > 0) ? 1 :
0;
27.        printk(KERN_INFO "Allocated TX Buffer: %zu bytes (total_length=%u
bytes)\n", g_dma_alloc_size_r, alloc_bytes);
28.    }
29.    // 假设 cmd == 1 代表分配 RX 缓冲区 (接收 FPGA, addr_w)
30.    else if (dma_operation.cmd == 1) {
31.        if (dma_info.addr_w.data_buf) {
32.            printk(KERN_ERR "RX Buffer already allocated!\n");
33.            spin_unlock(&dma_info.addr_r.lock);
34.            return -EBUSY;
35.        }
36.        g_dma_alloc_size_w = PAGE_ALIGN(alloc_bytes);
37.        dma_info.addr_w.data_buf = dma_alloc_coherent(&op_dev->dev,
38.                                                    g_dma_alloc_size_w,
```

`conv_conv` 模块是卷积计算的核心处理单元。它内部维护了一个 3x3 的输入数据滑动窗口缓存，通过对每 3 行输入数据进行移位寄存，形成 9 个数据点供乘法器使用。同时设计了乒乓缓存结构来存储卷积核权重，每个卷积核包含 3x3 个位置，每个位置又有 `CHA_PAR_IN` 个输入通道的权重数据。乘法器阵列按照输入通道并行度进行展开，每个周期可计算两个输出通道的乘积结果。随后通过多级加法树先累加 9 个点的乘积，再逐级累加所有输入通道的结果，最终得到每个输出通道的卷积输出。模块还通过延迟链对输入 `valid` 和 `last` 信号进行对齐，确保输出时序正确，支持 3x3 卷积和 1x1 卷积两种模式切换。

```

1. // conv_conv 核心：实现 3x3 和 1x1 卷积，支持输入/输出通道并行
2. module conv_conv #(parameter CHA_PAR_IN=16, CHA_PAR_OUT=8, INT=8, MUL_DELAY=4) (
3.     input clk, rst, start, type,
4.     input [CHA_PAR_IN*INT-1:0] weight_0~weight_8, weight_valid, weight_last,
5.     input [CHA_PAR_IN*INT-1:0] s_data_0, s_data_1, s_data_2, s_valid, s_last,
6.     output [CHA_PAR_OUT*(2*INT+4+$clog2(CHA_PAR_IN))-1:0] m_data,
7.     output m_valid, m_last
8. );
9. // 3 行输入数据缓存形成 3x3 窗口
10. reg [DATA_WIDTH-1:0] data[2:0][2:0];
11. always @(posedge clk) begin
12.     data[0][2] <= s_data_0; data[0][1] <= data[0][2]; data[0][0] <= data[0][1];
13.     data[1][2] <= s_data_1; data[1][1] <= data[1][2]; data[1][0] <= data[1][1];
14.     data[2][2] <= s_data_2; data[2][1] <= data[2][2]; data[2][0] <= data[2][1];
15. end
16.
17. // 权重乒乓缓存 (9 个点，每个点包含 CHA_PAR_IN 个通道)
18. reg [DATA_WIDTH-1:0] weight_mem_ping[CHA_PAR_OUT-1:0][2:0][2:0];
19. reg [DATA_WIDTH-1:0] weight_mem_pang[CHA_PAR_OUT-1:0][2:0][2:0];
20. reg w_ctl, r_ctl;
21. // 权重写入逻辑...
22.
23. // 乘法器阵列 (每周期计算 2 个输出通道)
24. generate for (i=0; i<CHA_PAR_IN; i=i+1)
25.     for (j=0; j<CHA_PAR_OUT/2; j=j+1) begin
26.         conv_mul #(.INT(INT), .MUL_DELAY(MUL_DELAY)) inst(
27.             .a(weight_data[i][j*2][r][c]), .b(weight_data[i][j*2+1][r][c]),
28.             .c(data[r][c][i*INT+:INT]), .a_c(mul[i][j*2][r][c]), .b_c(mul[i][j*2+1][r][c]));
29.     end
30. endgenerate

```

```

31.
32.    // 加法树 (3x3 点累加 + 输入通道累加)
33.    generate for (j=0; j<CHA_PAR_OUT; j=j+1)
34.        for (l=0; l<$clog2(CHA_PAR_IN); l=l+1)
35.            for (i=0; i<(CHA_PAR_IN>>(l+1)); i=i+1)
36.                add_cin[l][i][j] <= (l==0) ? add_33[i*2][j] + add_33[i*2+1][j] :
37.                    add_cin[l-1][i*2][j] + add_cin[l-
1][i*2+1][j];
38.    endgenerate
39.
40.    // 输出及 valid/last 延迟控制
41.    assign m_data[j] = add_cin[$clog2(CHA_PAR_IN)-1][0][j];
42.    assign m_valid = (type ? data_valid_d[DELAY-2-1] : data_valid_d[DELAY-1]) &&
(delay_cnt==2);
43.    assign m_last = type ? data_last_d[DELAY-2-1] : data_last_d[DELAY-1];
44. endmodule
45.

```

conv_weight 模块负责从外部 DMA 读取权重和偏置数据并缓存供给卷积模块。它包含一个状态机控制器，按需向 DMA 发起读请求，读取指定基地址和长度的权重数据。模块内部使用 9 个独立的 FIFO 分别存储卷积核中 9 个位置的权重，写入时轮流填充这 9 个 FIFO，实现权重的空间分解。读出时根据卷积类型（3x3 或 1x1）采用不同的读取策略：3x3 模式同时读出 9 个 FIFO 的数据，1x1 模式仅读出中心位置。偏置数据则单独存储在一个 SPRAM 中，独立输出通道供后续加法使用。整个模块通过乒乓机制和流控信号（ready/valid）实现与上下游模块的高效数据交换。

```

1. // conv_weight 核心：从 DMA 读取权重/偏置，乒乓缓存后供给卷积模块
2. module conv_weight #(parameter CHA_PAR_IN=16, CHA_PAR_OUT=8, DATA_WIDTH=128) (
3.     input clk, rst, start, type, stride,
4.     input [31:0] base_addr, weight_sum,
5.     input [DATA_WIDTH-1:0] s_data, s_valid, s_last, output s_ready,
6.     output [DATA_WIDTH-1:0] m_data_0~m_data_8, output m_valid, m_last, input m_req,
7.     output [31:0] bias, output bias_valid, input bias_req,
8.     output [31:0] cmd_addr, cmd_len, cmd_valid, input cmd_ready
9. );
10.    // 状态机：IDLE → WADDR → WDATA → W_WAIT → RDATA
11.    localparam IDLE=0, WADDR=1, WDATA=2, W_WAIT=3;
12.    reg [1:0] state;
13.
14.    // DMA 读请求（地址、长度）
15.    assign cmd_addr = data_addr;

```

```

16.    assign cmd_len = weight_len_r;
17.    assign cmd_valid = cmd_valid_r;
18.
19.    // 9 个 FIFO 存储 9 个位置的权重
20.    wire [DATA_WIDTH-1:0] rdata [8:0];
21.    generate for (i=0; i<9; i=i+1)
22.        sync_fifo_D #(.DATA_WIDTH(DATA_WIDTH), .DATA_DEPTH(512)) fifo(
23.            .wr_en(wen[i]), .rd_en(ren[i]), .din(s_data), .dout(rdata[i]),
24.            .full(full[i]), .empty(empty[i]));
25.    endgenerate
26.
27.    // 权重写入 (轮流写 9 个 FIFO)
28.    assign wen[i] = (w_ram_cnt == i+1) && s_valid && s_ready;
29.
30.    // 权重读出 (根据 type 选择 1x1 或 3x3 模式)
31.    assign ren_all = (state==RDATA) && m_ready && ~empty[0];
32.    assign m_valid = ren_all;
33.    assign m_last = (raddr == CHA_PAR_OUT-1);
34.
35.    // 偏置 RAM (独立存储)
36.    spram #(.DP(BIAS_DEPTH), .DW(BIAS_WIDTH)) bias_ram(...);
37.    assign bias = rdata_bias;
38.    assign bias_valid = ren_bias;
39. endmodule
40.

```

3. 仿真测试

在项目的实际工作过程中，数据调度与指令传递效果可以通过波形仿真图展现。图 12 展示了权重缓存读取在实际操作中的表现，实现了从 DMA 读取卷积权重和偏置数据、分 9 个 RAM 缓存 3x3 卷积核的权重、按输入/输出通道并行度组织数据、并响应下游计算单元的请求依次输出权重和偏置的功能。

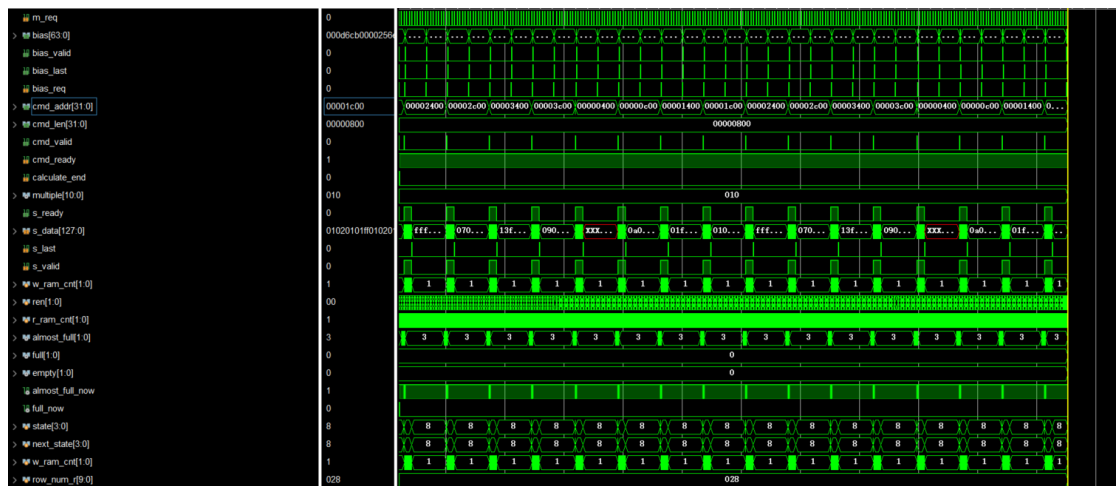


图 12-权重缓存读取波形图

图 13 展示了单层卷积的结果输出表现，实现了将卷积计算结果通过乒乓缓存（ping/pang）交替写入，按输出通道并行度重组数据，并通过输出选择器（out_switch）根据启动信号选择不同来源（如卷积、SPPF、上采样等）的数据，最终经 FIFO 和 DMA 接口输出到外部存储的功能。

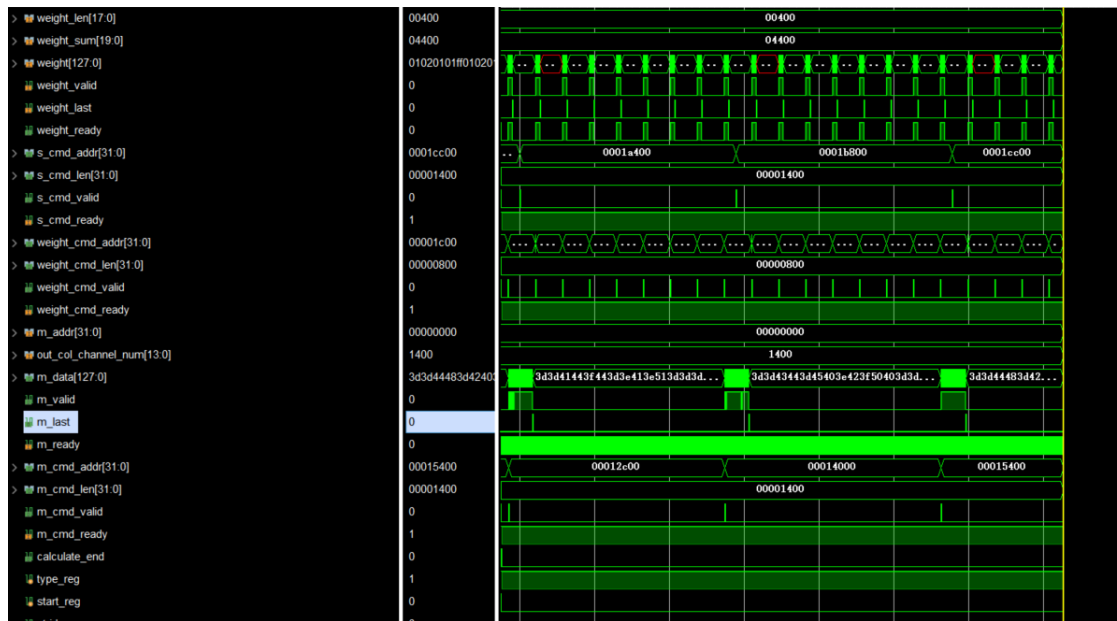


图 13-卷积结果输出波形图

图 14 展示了 DDR 数据的读取表现。

